

Diplôme National D'ingénieur

2025 / 2026

Spécialité : Genie Informatique
Data Science et Intelligence Artificielle

Titre du sujet :

**Développement d'un système d'audit réglementaire basé sur l'intelligence artificielle
pour le cadre réglementaire Simplex de l'immobilier portugais.**

Réalisé par : Mouissaoui Leila

Encadrant Universitaire : Abida Mounir

Encadrant Professionnel : Pieter Paul

Dedicace

A mes parents,

*pour leur amour inconditionnel, leur soutien constant et leurs sacrifices tout au long
de mon parcours académique. Rien de tout cela n'aurait été possible sans vous.*

A mon frère,

*pour sa présence, ses encouragements et sa confiance en moi à chaque étape de ce
chemin.*

A mes sœurs,

*pour leur soutien précieux et leur affection qui m'ont donné la force d'avancer dans
les moments de doute.*

A tous ceux qui ont cru en moi

et qui m'ont accompagné(e) jusqu'à l'aboutissement de ce projet.

Remerciements

Au terme de ce travail, qui marque l'aboutissement de plusieurs années de formation, je tiens à exprimer ma profonde **gratitude** à toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce **projet de fin d'études**.

Je tiens tout d'abord à remercier chaleureusement mon **encadrant académique**, **Abida Mounir**, pour sa disponibilité sans faille, ses conseils avisés et son suivi rigoureux tout au long de ce projet. Sa rigueur scientifique, sa patience et ses remarques constructives m'ont permis de progresser constamment et de mener à bien ce travail dans les meilleures conditions.

Mes sincères remerciements vont également à mon **encadrant professionnel** au sein de Remax Wise (Portugal), **Pieter Paul Castelein**, pour son accompagnement précieux, la confiance qu'il m'a accordée dès le début du stage, ainsi que pour les nombreux échanges enrichissants qui ont guidé les choix techniques et stratégiques de ce travail. Sa disponibilité et son expertise ont été des atouts majeurs dans la concrétisation de ce projet.

Je souhaite également exprimer ma reconnaissance à l'ensemble de l'équipe de Remax Wise (Portugal) pour leur accueil chaleureux, leur esprit d'équipe et leur disponibilité tout au long de cette expérience professionnelle enrichissante. Ces échanges quotidiens ont grandement contribué à l'approfondissement de mes connaissances techniques et à mon intégration au sein du monde professionnel.

Mes remerciements s'adressent aussi à l'ensemble du corps professoral et administratif d'École Supérieure Privée d'Ingénieurs pour la qualité de la formation dispensée durant ce cursus, ainsi que pour les efforts constants déployés afin d'assurer les meilleures conditions d'étude et d'encadrement.

Enfin, je tiens à exprimer toute ma gratitude aux **membres du jury** pour l'honneur

qu'ils me font en acceptant d'évaluer ce travail. Leurs remarques et leurs suggestions seront précieuses pour l'enrichissement de cette étude.

À toutes celles et ceux qui, d'une manière ou d'une autre, ont apporté leur pierre à l'édifice de ce projet, je vous adresse mes plus sincères remerciements.

Résumé

Le marché **immobilier** portugais est soumis à un ensemble de règles complexes. Ces règlements, qui comprennent le programme de simplification administrative **Simplex** et des lois propres aux villes, rendent l'évaluation de la **conformité légale** d'un bien très longue, coûteuse et difficile à comprendre pour les non-spécialistes.

Ce rapport explique la conception et la mise en œuvre de PropIQ, un assistant juridique intelligent destiné au secteur immobilier en Portugal. Ce projet a été réalisé pendant un stage de fin d'études chez Remax Wise (Portugal). La solution comporte deux éléments principaux :

- un **chatbot juridique** basé sur une architecture de **Retrieval-Augmented Generation (RAG)**. Ce chatbot répond en langage naturel, avec des justifications et une trace des sources. Il peut **analyser des images et des fichiers PDF**, et même répondre par la voix. Les utilisateurs peuvent poser leurs questions à voix haute ;
- un **module qui évalue la qualité de vie** en se basant sur un score calculé à partir de plusieurs critères comme les transports, les hôpitaux, les écoles, les commerces et les espaces verts, obtenus via **OpenStreetMap** et l'**API Overpass**, ainsi que la qualité de l'air, obtenue via l'**Open-Meteo Air Quality API**.

Le projet a suivi la méthodologie **CRISP-DM**, de la phase de compréhension métier jusqu'au déploiement. La solution s'appuie sur les technologies suivantes :

- une architecture backend **FastAPI** ;
- une base vectorielle **Pinecone** pour des recherches sémantiques ;
- un accès à plusieurs modèles de langage via **OpenRouter** ;

- une authentification **Firestore** ;
- une application mobile développée avec **Flutter**.

Un suivi expérimental a été effectué via **MLflow**, et une supervision via la stack **Elasticsearch/Kibana** a été mise en place afin de garantir la fiabilité et la traçabilité du système une fois en production.

Une phase de **tests et de validation**, détaillée au chapitre 6, a permis d'évaluer la pertinence du chatbot juridique, la robustesse du module d'analyse multimodale, ainsi que la cohérence de l'indicateur de qualité de vie. Les résultats montrent que le système est capable de fournir des réponses juridiques pertinentes et justifiées, et qu'il propose un indicateur de qualité de vie en accord avec les données disponibles. Le système respecte également les exigences en matière de sécurité, de scalabilité et de reproductibilité attendues d'un produit LegalTech.

Mots-clés : NLP, RAG, grands modèles de langage, conformité réglementaire, immobilier, cadre Simplex, analyse multimodale, reconnaissance et synthèse vocale, FastAPI, Flutter.

Abstract

The Portuguese **real estate** market is subject to a complex set of rules. These regulations, which include the **Simplex** administrative simplification program and city-specific laws, make assessing a property's **legal compliance** very time-consuming, costly, and difficult for non-specialists to understand.

This report explains the design and implementation of PropIQ, an intelligent legal assistant for the real estate sector in Portugal. This project was carried out during a final-year internship at Remax Wise (Portugal). The solution comprises two main components:

- a **legal chatbot** based on a **Retrieval-Augmented Generation (RAG)** architecture. This chatbot answers in natural language, with justifications and source traceability. It can **analyze images and PDF files**, and even respond by voice. Users can ask their questions out loud;
- a **quality-of-life assessment module**, based on a score computed from several criteria such as transport, hospitals, schools, commercial amenities, and green spaces, obtained via **OpenStreetMap** and the **Overpass API**, together with air quality, obtained via the **Open-Meteo Air Quality API**.

The project followed the **CRISP-DM** methodology, from the business understanding phase through to deployment. The solution relies on the following technologies:

- a **FastAPI** backend architecture;
- a **Pinecone** vector database for semantic search;
- access to multiple large language models via **OpenRouter**;

- **Firestore** authentication;
- a mobile application developed with **Flutter**.

Experiment tracking was carried out via **MLflow**, and monitoring via the **Elasticsearch/Kibana** stack was put in place to ensure the reliability and traceability of the system once in production.

A **testing and validation** phase, detailed in Chapter 6, made it possible to evaluate the relevance of the legal chatbot, the robustness of the multimodal analysis module, as well as the consistency of the quality-of-life indicator. The results show that the system is able to provide relevant and well-justified legal answers, and that it offers a quality-of-life indicator consistent with the available data. The system also meets the security, scalability, and reproducibility requirements expected of a LegalTech product.

Keywords: NLP, RAG, large language models, regulatory compliance, real estate, Simplex framework, multimodal analysis, speech recognition and synthesis, FastAPI, Flutter.

Contents

Dedicace	i
Remerciements	ii
Résumé	iv
Abstract	vi
Table des matieres	xi
Liste des figures	xiii
Liste des tableaux	xiv
Liste des abréviations	xv
Introduction générale	1
1 État de l’art	5
1.1 Introduction	5
1.2 Organisme d’accueil	6
1.3 Cadre réglementaire portugais	7
1.3.1 Programme Simplex et réglementation immobilière	7
1.3.2 Corpus réglementaire utilisé par PropIQ	8
1.4 Étude des solutions existantes	10
1.4.1 Synthèse comparative	10
1.4.2 Outils de recherche juridique basés sur l’IA	11
1.4.3 Plateformes immobilières portugaises	12
1.4.4 Positionnement de PropIQ	12
1.5 Méthodologie CRISP-DM	13

1.6	Conclusion	15
2	Compréhension des métiers et des données	17
2.1	Introduction	17
2.2	Compréhension du métier	17
2.2.1	Contexte métier et problématique	18
2.2.2	Objectifs métier et objectifs Data Science	18
2.2.3	Identification des acteurs	19
2.2.4	Besoins fonctionnels	20
2.2.5	Besoins non fonctionnels	22
2.2.6	Product Backlog	24
2.2.7	Critères de succès du projet	26
2.3	Compréhension des données	26
2.3.1	Sources de données	26
2.3.2	Données juridiques	27
2.3.3	Données géographiques et environnementales	28
2.3.4	Exploration et qualité des données	29
2.4	Conclusion	30
3	Préparation des données	32
3.1	Introduction	32
3.2	Préparation des données juridiques	32
3.2.1	Collecte des textes depuis le Diário da República	33
3.2.2	Nettoyage et prétraitement textuel	35
3.2.3	Segmentation des textes (chunking)	36
3.2.4	Génération des embeddings vectoriels	36
3.2.5	Indexation dans la base vectorielle	37
3.3	Préparation des données géographiques et environnementales	37
3.3.1	Géocodage des adresses	38
3.3.2	Collecte et normalisation des points d'intérêt	38
3.3.3	Normalisation des données de qualité de l'air	39
3.4	Conclusion	40
4	Modélisation et évaluation	41

4.1	Introduction	41
4.2	Modélisation	41
4.2.1	Modélisation du pipeline RAG	41
4.2.2	Modèles d'intelligence artificielle mobilisés	43
4.2.3	Modélisation de l'indice de qualité de vie	49
4.3	Évaluation des modèles de langage	51
4.3.1	Protocole et environnement d'évaluation	51
4.3.2	Métriques retenues	52
4.3.3	Résultats comparatifs	53
4.3.4	Analyse et choix du modèle	53
4.4	Conclusion	54
5	Déploiement	56
5.1	Introduction	56
5.2	Choix technologiques	57
5.2.1	Flutter	57
5.2.2	FastAPI	58
5.2.3	Firestore Authentication	58
5.2.4	FAISS	59
5.2.5	Pinecone	59
5.2.6	OpenRouter	60
5.2.7	SQLite	60
5.2.8	Stripe	61
5.2.9	Docker	62
5.2.10	GitHub Actions	62
5.2.11	MLflow	63
5.2.12	Elasticsearch et Kibana	63
5.2.13	Ngrok	64
5.3	Comparaison entre FAISS et Pinecone	65
5.4	Diagrammes de conception et d'architecture	66
5.4.1	Diagrammes UML	66
5.4.2	Architecture globale	69
5.4.3	Architecture du pipeline NLP et RAG	70

5.5	Pipeline MLOps	71
5.6	Déploiement technique	77
5.7	Réalisation du backend FastAPI	77
5.7.1	Organisation générale	77
5.7.2	Pipeline RAG et accès multi-LLM	80
5.7.3	Système de crédits et quotas	81
5.7.4	Transcription vocale	82
5.7.5	Authentification et sécurité	83
5.7.6	Protection des données et aspects réglementaires	84
5.8	Interfaces graphiques de l'application	85
5.8.1	Application mobile Flutter	85
5.9	Conclusion	89
6	Tests et validation	90
6.1	Introduction	90
6.2	Plan de validation	90
6.3	Tests unitaires	91
6.4	Tests boîte blanche	92
6.5	Tests boîte noire	94
6.6	Tests de charge et de performance	96
6.7	Tests de sécurité	98
6.8	Protocoles et conditions expérimentales des modules IA	99
6.9	Discussion critique	101
6.10	Perspectives	102
6.11	Conclusion	102
	Conclusion générale	103
	Références bibliographiques	106

List of Figures

1.1	Logo de REMAX Wise	6
1.2	Les différentes phases de la méthodologie CRISP-DM [1]	14
4.1	Architecture du pipeline RAG de PropIQ	43
5.1	Logo Flutter	57
5.2	Logo FastAPI	58
5.3	Logo Firebase	58
5.4	Logo FAISS	59
5.5	Logo Pinecone	59
5.6	Logo OpenRouter	60
5.7	Logo SQLite	60
5.8	Logo Stripe	61
5.9	Logo Docker	62
5.10	Logo GitHub Actions	62
5.11	Logo MLflow	63
5.12	Logos Elasticsearch et Kibana	63
5.13	Logo Ngrok	64
5.14	Diagramme de cas d'utilisation de PropIQ	66
5.15	Diagramme de classes de PropIQ	68
5.16	Architecture globale de la solution PropIQ	70
5.17	Architecture du pipeline NLP et RAG de PropIQ	70
5.18	Architecture de la couche MLOps	73
5.19	Documentation Swagger de l'API PropIQ	78
5.20	Routes de gestion de l'historique, des crédits et de l'analyse de qualité de vie	79
5.21	Routes d'évaluation, de modèles et de gestion des abonnements et paiements	80

5.22	Tableau de bord Stripe pour les transactions des abonnements Premium	82
5.23	Pipeline Voice-to-RAG	83
5.24	Console Firebase – gestion des utilisateurs et des métadonnées	84
5.25	Exposition temporaire de l’API locale avec Ngrok	86
5.26	Captures d’écran de l’application mobile PropIQ – parcours utilisateur et chatbot (1/2)	87
5.27	Captures d’écran de l’application mobile PropIQ – qualité de vie et paramètres (2/2)	88
6.1	Rapport de couverture de code généré par pytest-cov (seuil de 80 % atteint)	92
6.2	Résultat de l’exécution des tests boîte blanche (pytest -v, 22 tests) . . .	93
6.3	Test boîte noire : envoi d’une question juridique et réponse du chatbot dans l’application Flutter	95
6.4	Tableau de bord Kibana : latence et taux d’erreur de l’API en conditions de charge	98
6.5	Résultat de l’analyse statique de sécurité avec bandit (make security)	99
6.6	Interface MLflow : comparaison des cinq modèles LLM sur la pertinence des réponses et la diversité des sources juridiques mobilisées	100

List of Tables

1.1	Informations générales sur l'organisme d'accueil	7
1.2	Principales catégories de documents réglementaires utilisées dans PropIQ	9
1.3	Comparaison des solutions existantes avec PropIQ	11
2.1	Correspondance entre les objectifs métier et les objectifs Data Science	19
2.2	Besoins fonctionnels de PropIQ	21
2.3	Besoins non fonctionnels de PropIQ	23
2.4	Product Backlog de PropIQ	25
2.5	Sources de données utilisées par PropIQ	27
3.1	Catégories thématiques du corpus juridique (1/2)	33
3.2	Catégories thématiques du corpus juridique (2/2)	34
4.1	Pondération des critères de l'indice de qualité de vie	49
4.2	Comparaison des modèles de langage selon le temps de réponse, la pertinence et les hallucinations détectées	53
5.1	Comparaison entre FAISS et Pinecone	65
6.1	Synthèse des tests boîte blanche	93
6.2	Paramètres et indicateurs des tests de charge	97

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface (interface de programmation applicative)
ASGI	Asynchronous Server Gateway Interface
BDD	Base de données
CI/CD	Continuous Integration / Continuous Deployment (intégration et déploiement continus)
CRISP-DM	Cross-Industry Standard Process for Data Mining
DevSecOps	Development, Security and Operations
DNS	Domain Name System
FAISS	Facebook AI Similarity Search
GQA	Grouped-Query Attention (attention par groupes)
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (grand modèle de langage)
MLOps	Machine Learning Operations
MoE	Mixture-of-Experts (mélange d'experts)
MoSCoW	Must have, Should have, Could have, Won't have (méthode de priorisation)
NLP	Natural Language Processing (traitement automatique du langage naturel)

Abréviation	Signification
OCR	Optical Character Recognition (reconnaissance optique de caractères)
PDF	Portable Document Format
PEP8	Python Enhancement Proposal 8
PFE	Projet de Fin d'Études
POI	Point(s) d'intérêt
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RoPE	Rotary Positional Encoding (encodage positionnel rotatif)
SaaS	Software as a Service
SDK	Software Development Kit
STT	Speech-to-Text
TTS	Text-to-Speech
UML	Unified Modeling Language

Introduction générale

Contexte général

Le secteur **immobilier** est l'un des piliers de l'économie portugaise, surtout dans les zones qui attirent beaucoup de touristes et de résidents, comme la région de Setúbal. La demande pour les biens immobiliers augmente constamment, à la fois auprès des habitants du pays et des acheteurs étrangers. Cela fait du marché immobilier portugais un secteur en pleine croissance, mais aussi de plus en plus difficile à comprendre.

Acheter un bien immobilier au Portugal implique de respecter un ensemble de règles complexes. Ce cadre réglementaire repose sur un programme appelé **Simplex**, qui vise à simplifier les démarches administratives, mais il s'ajoute à des lois spécifiques à chaque commune. Ces textes légaux, souvent techniques et en constante évolution, rendent l'évaluation de la **conformité** d'un bien difficile pour les particuliers, les agents immobiliers et même certains professionnels du droit.

En dehors des aspects légaux, l'achat d'un bien immobilier dépend également de critères liés à l'environnement immédiat du bien : qualité des transports, présence d'écoles, d'établissements de santé, de commerces et d'espaces verts à proximité, ainsi que la qualité de l'air. Ces informations sont dispersées entre de nombreuses sources géographiques et environnementales, et difficiles à synthétiser pour un acheteur non spécialiste. Cela rend la décision d'achat encore plus compliquée.

C'est dans ce contexte qu'a été conçu ce projet de fin d'études, mené au sein de l'agence Remax Wise (Portugal). L'objectif est de créer une plateforme intelligente capable d'automatiser l'audit réglementaire d'un bien immobilier et d'évaluer son environnement de vie. Pour cela, on s'appuie sur les dernières avancées du **traitement automatique du langage naturel** et des **grands modèles de langage**.

Problématique

Le problème principal de ce projet est de savoir comment permettre à un utilisateur, qu'il soit expert ou non, d'obtenir une évaluation fiable et rapide de la **conformité légale** d'un bien immobilier au Portugal. Cela doit également inclure une vision complète de la qualité de vie associée à ce bien, sans avoir à consulter manuellement de nombreux textes juridiques et sources de données géographiques et environnementales dispersées.

Actuellement, cette évaluation repose principalement sur l'expertise humaine. Les notaires, les avocats et les agents immobiliers doivent consulter individuellement les textes du programme Simplex, les règlements municipaux et les documents d'urbanisme propres à chaque commune. Ce processus est non seulement long et coûteux, mais il comporte également le risque d'erreur ou d'omission, notamment lorsque la réglementation évolue ou diffère d'une commune à l'autre.

Ce problème soulève plusieurs défis techniques. Le premier concerne la structuration et la mise en forme de textes juridiques volumineux et variés, rédigés dans un langage administratif complexe. Le second défi est de concevoir un moteur de **recherche** fiable qui puisse alimenter une génération de réponses enrichies tout en limitant les erreurs. Le troisième défi est de garantir que les réponses générées soient traçables et justifiables, ce qui est essentiel pour inspirer confiance aux utilisateurs dans un domaine comme le droit immobilier. Enfin, il faut intégrer des sources de données variées dans un système unique et exploitable.

Comme il n'existe pas d'outil intégré pour répondre à ces défis réglementaires et environnementaux, ce projet vise à développer une solution automatisée, fiable et accessible. L'objectif est de réduire significativement le temps et le coût associés à l'évaluation d'un bien immobilier au Portugal.

Objectifs du projet

Ce projet vise à atteindre plusieurs objectifs importants :

- développer un **chatbot juridique** qui utilise une architecture **NLP** et **RAG**.

Cela signifie que le chatbot pourra répondre aux questions de manière naturelle et faire référence aux textes applicables de manière explicite ;

- créer un module qui évalue la **qualité de vie**. Ce module calculera un score en fonction des lieux importants qui se trouvent près de l'adresse d'un bien immobilier, notamment les transports, les écoles, les hôpitaux et les commerces ;
- rendre cette solution accessible à tous en la transformant en une **API FastAPI**. Cette API sera contenue dans un conteneur et pourra être utilisée via une application mobile, avec la sécurité, le suivi et la supervision intégrés pour garantir une expérience fiable et sécurisée.

Plan du rapport

Ce rapport est organisé de manière logique en six chapitres, structurés selon la méthodologie CRISP-DM adoptée pour ce projet.

Le premier chapitre présente l'**état de l'art**. Il commence par introduire l'organisme d'accueil et le cadre réglementaire Simplex, puis il examine les solutions existantes, notamment les outils de recherche juridique fondés sur l'IA générative et les plateformes immobilières portugaises. Ce chapitre se termine par une présentation de la méthodologie CRISP-DM adoptée pour structurer la suite du projet.

Le deuxième chapitre couvre la **compréhension des métiers et des données**. Il traduit la problématique métier en objectifs data science, identifie les acteurs du système et détaille les besoins fonctionnels et non fonctionnels sous forme de backlogs, complétés par le product backlog exprimé en epics et user stories. Ce chapitre présente ensuite les sources de données mobilisées par le projet, qu'il s'agisse du corpus juridique ou des données géographiques et environnementales utilisées pour le calcul de l'indice de qualité de vie.

Le troisième chapitre porte sur la **préparation des données**. Il détaille la collecte des textes juridiques depuis le Diário da República, leur nettoyage, leur segmentation en chunks et leur vectorisation en vue de l'indexation dans la base vectorielle Pinecone. Il présente également la préparation des données géographiques et environnementales,

incluant le géocodage des adresses ainsi que la collecte et la normalisation des points d'intérêt et des données de qualité de l'air.

Le quatrième chapitre couvre la **modélisation et l'évaluation** des modèles d'intelligence artificielle mobilisés. Il présente le pipeline RAG ainsi que les modèles retenus selon leur usage – traduction, transcription vocale, raisonnement juridique et analyse multimodale. Ce chapitre détaille ensuite le protocole et les résultats de l'évaluation comparative des modèles de raisonnement juridique, réalisée à l'aide du suivi expérimental MLflow, ainsi que l'analyse ayant conduit au choix du modèle retenu en production.

Le cinquième chapitre décrit le **déploiement** de la solution. Il justifie le choix des technologies retenues et présente la comparaison entre FAISS et Pinecone ayant motivé le choix définitif de la base vectorielle. Il détaille les diagrammes de conception (cas d'utilisation, classes), l'architecture globale du système, l'architecture du pipeline NLP et RAG, ainsi que la couche MLOps mise en place selon sept phases clés. Ce chapitre présente également la réalisation technique du backend FastAPI, l'authentification et la sécurité du système, et se termine par la présentation des interfaces graphiques de l'application mobile Flutter.

Enfin, le sixième chapitre présente les **tests et la validation** du système. Il expose le plan de validation, puis détaille les tests unitaires, les tests boîte blanche et boîte noire, les tests de charge et de performance, ainsi que les tests de sécurité. Les protocoles et conditions expérimentales des modules d'intelligence artificielle sont également décrits, avant une discussion critique des résultats et la présentation des perspectives d'évolution du projet.

Une conclusion générale clôturera ce rapport, en synthétisant les apports du travail réalisé et en ouvrant sur des perspectives d'évolution future.

CHAPITRE 1

État de l’art

1.1 Introduction

Ce chapitre présente le contexte général dans lequel le projet PropIQ a été réalisé. Il commence par une présentation de l’organisme d’accueil et de son activité dans le domaine de l’immobilier au Portugal.

Une attention particulière est ensuite accordée au cadre réglementaire portugais lié au programme Simplex ainsi qu’aux règles d’urbanisme applicables à Setúbal. Ces éléments sont importants pour le projet, car PropIQ a été conçu pour aider les utilisateurs à mieux comprendre les règles qui peuvent s’appliquer à un bien immobilier selon sa localisation, son type et son usage.

Le chapitre présente également une étude de plusieurs solutions déjà disponibles sur le marché. Cette comparaison concerne à la fois des outils de recherche juridique utilisant l’intelligence artificielle et des plateformes immobilières portugaises. L’objectif est d’identifier leurs principales fonctionnalités ainsi que leurs limites par rapport aux besoins couverts par PropIQ.

Enfin, la méthodologie CRISP-DM adoptée pour organiser les différentes étapes du projet est présentée. Elle permet de suivre une démarche structurée, depuis la compréhension du besoin jusqu’à l’évaluation et au déploiement de la solution [1].

1.2 Organisme d'accueil



Figure 1.1: Logo de REMAX Wise

Le projet de fin d'études a été réalisé au sein de **Remax Wise (Portugal)**, une agence appartenant au réseau REMAX et active dans le secteur immobilier portugais.

L'entreprise accompagne différents profils de clients dans leurs projets immobiliers, notamment pour l'achat, la vente et la location de biens. Son activité concerne aussi bien des clients locaux que des clients internationaux souhaitant investir ou acquérir un bien au Portugal.

Dans ce contexte, l'utilisation de solutions numériques peut faciliter l'accès à l'information et améliorer l'accompagnement des clients. C'est dans cette perspective que le projet PropIQ a été proposé. La solution vise à fournir un outil d'aide à la décision capable de centraliser différentes informations utiles à l'analyse d'un bien immobilier.

Dans sa première version, PropIQ se concentre sur le contexte de Setúbal et sur les principales informations réglementaires et environnementales pouvant influencer une décision immobilière. À terme, l'approche pourra être adaptée à d'autres contextes réglementaires et géographiques.

“

Table 1.1: Informations générales sur l’organisme d’accueil

Information	Détail
Nom de l’entreprise	REMAX Wise
Secteur d’activité	Immobilier
Activités principales	Achat, vente et location de biens immobiliers
Type de clientèle	Clients nationaux et internationaux
Pays d’activité	Portugal
Téléphone	+351 214 213 245
Adresse e-mail	protecaodados.wiseremax.pt

“

1.3 Cadre réglementaire portugais

1.3.1 Programme Simplex et réglementation immobilière

Le programme **Simplex** constitue une initiative de modernisation et de simplification administrative et législative mise en place par l’État portugais. Son objectif est notamment de simplifier les relations entre les citoyens, les entreprises et l’administration publique, tout en améliorant l’efficacité des services publics [2].

Dans le domaine de l’urbanisme, le programme Simplex a notamment contribué à une réforme des procédures administratives liées à l’urbanisation et à la construction. Le *Decreto-Lei n.º 10/2024*, publié le 8 janvier 2024, s’inscrit dans cette démarche de simplification des procédures administratives en matière d’aménagement du territoire et d’urbanisme [3].

Cependant, les règles applicables à un bien immobilier ne dépendent pas uniquement du cadre national. Les dispositions nationales sont complétées par des instruments de gestion territoriale et des réglementations municipales qui peuvent définir les conditions d’utilisation des terrains, les possibilités de construction ainsi que différentes contraintes applicables à un bien.

Dans le cas de Setúbal, le *Plano Diretor Municipal* (PDM) constitue notamment une source importante pour l’analyse du contexte territorial et urbanistique. La révision du PDM de Setúbal a été approuvée en septembre 2024 et publiée au *Diário da República* par l’*Aviso n.º 2639/2025/2* du 28 janvier 2025 [4]. Le PDM en vigueur comprend notamment des informations relatives à la classification et à la qualification du sol, aux risques, à la structure écologique, au patrimoine ainsi qu’aux servitudes et restrictions d’utilité publique [4].

Cette coexistence entre les règles nationales et les réglementations locales rend l’analyse d’un bien immobilier plus complexe. Pour obtenir une réponse pertinente, il est nécessaire de prendre en compte plusieurs éléments, notamment la localisation du bien, son type, son usage et sa situation administrative.

Une vérification réalisée uniquement de manière manuelle peut donc demander du temps et nécessiter la consultation de plusieurs documents. C’est l’un des besoins auxquels PropIQ cherche à répondre en utilisant l’intelligence artificielle et la recherche augmentée par génération (*Retrieval-Augmented Generation*, RAG).

1.3.2 Corpus réglementaire utilisé par PropIQ

Le module de recherche juridique de PropIQ s’appuie sur un corpus de documents réglementaires sélectionnés en fonction des besoins du projet. Ces documents servent de base aux recherches effectuées par le système et permettent au modèle de génération de produire des réponses accompagnées de références provenant des sources consultées.

Le corpus regroupe plusieurs niveaux de documents. Il comprend notamment des mesures liées au programme Simplex, des textes législatifs nationaux ainsi que des documents réglementaires et d’urbanisme propres à Setúbal.

Le tableau 1.2 présente les principales catégories de documents considérées dans le projet.

““

Table 1.2: Principales catégories de documents réglementaires utilisées dans PropIQ

Niveau	Référence	Date / ver- sion	Autorité	Périmètre
Programme de simplification	Programa Simplex [2]	Selon les mesures	Gouvernement portugais	National
Texte législatif	Decreto-Lei n.º 10/2024 – mesures de simplification en matière d’urbanisme [3]	Janvier 2024	Gouvernement portugais	National
Document d’urbanisme	Plano Diretor Municipal de Setúbal – Aviso n.º 2639/2025/2 [4]	Janvier 2025	Câmara Municipal de Setúbal	Municipal
Réglementation municipale	Réglementation d’édification et d’urbanisme du Município de Setúbal [5]	Version en vigueur	Câmara Municipal de Setúbal	Municipal

””

L’intérêt de cette organisation est de permettre à PropIQ de rechercher les informations pertinentes dans plusieurs sources au lieu de se limiter à un seul document. Lorsqu’un utilisateur pose une question, le système recherche les passages les plus proches de sa demande dans le corpus, puis les transmet au modèle de langage afin de générer une réponse contextualisée.

Cette approche est particulièrement importante dans le domaine juridique, où une réponse doit pouvoir être vérifiée à partir des documents utilisés. Les références récupérées par le système permettent ainsi d’améliorer la traçabilité des réponses fournies par PropIQ.

1.4 Étude des solutions existantes

Afin de mieux positionner PropIQ, plusieurs solutions existantes ont été étudiées. La comparaison porte sur deux catégories principales.

La première catégorie regroupe des outils de recherche juridique utilisant l’intelligence artificielle, tels que Lexis+, Westlaw et CoCounsel. Ces solutions proposent des fonctionnalités avancées de recherche et d’analyse juridique assistées par l’intelligence artificielle [6–8], mais elles restent principalement généralistes.

La seconde catégorie correspond aux plateformes immobilières utilisées au Portugal, notamment Idealista.pt et Imovirtual.pt. Ces plateformes facilitent principalement la recherche et la consultation d’annonces immobilières [9, 10].

L’objectif de cette étude n’est donc pas de comparer uniquement les fonctionnalités techniques des différentes solutions, mais de vérifier dans quelle mesure elles répondent aux besoins spécifiques identifiés pour PropIQ.

1.4.1 Synthèse comparative

Le tableau 1.3 présente les principales différences entre les solutions étudiées et PropIQ.

““

Table 1.3: Comparaison des solutions existantes avec PropIQ

Critère	Recherche juridique IA	Plateformes immobilières	PropIQ
Spécialisation du cadre Simplex et de Setúbal	✗	✗	✓
Chatbot juridique basé sur le RAG	✓	✗	✓
Réponses accompagnées de références	~	N/A	✓
Analyse de documents immobiliers	~	✗	✓
Indicateur de qualité de vie	✗	~	✓
Analyse adaptée à un acheteur étranger	~	~	✓

““

1.4.2 Outils de recherche juridique basés sur l’IA

Les solutions comme Lexis+, Westlaw et CoCounsel montrent l’intérêt croissant de l’intelligence artificielle dans le domaine juridique. Elles permettent notamment de rechercher des informations dans de grands volumes de documents et de générer ou synthétiser des réponses à partir de requêtes formulées en langage naturel [6–8].

Cependant, ces outils sont conçus pour répondre à des besoins juridiques généraux. Ils ne sont pas spécifiquement construits autour du cadre réglementaire immobilier de Setúbal ni autour du programme Simplex considéré dans le présent projet.

Une autre limite concerne la génération automatique des réponses. Comme pour les autres systèmes basés sur des modèles de langage, le risque d’hallucination ne peut pas être totalement éliminé [11, 12]. Il est donc important de relier les réponses aux

documents réellement récupérés par le système.

PropIQ reprend le principe du RAG, mais l’applique à un corpus spécialement préparé pour le contexte du projet. Le système cherche ainsi à fournir des réponses plus adaptées au domaine immobilier étudié tout en conservant une trace des sources utilisées.

1.4.3 Plateformes immobilières portugaises

Des plateformes telles qu’Idealista.pt et Imovirtual.pt sont largement orientées vers la recherche de biens immobiliers. Elles permettent aux utilisateurs de consulter des annonces selon différents critères comme le prix, la localisation, la surface ou le type de bien [9, 10].

Ces plateformes sont utiles pour trouver un bien, mais leur objectif principal reste la mise en relation entre l’offre et la demande immobilière. Elles ne proposent pas une analyse réglementaire approfondie permettant de vérifier automatiquement les contraintes applicables à un bien donné.

Elles ne regroupent pas non plus, dans une même analyse, les informations réglementaires, les documents fournis par l’utilisateur et les indicateurs liés à la qualité de vie du quartier.

PropIQ cherche donc à compléter ce type de plateforme en ajoutant une couche d’analyse et d’aide à la décision. L’utilisateur peut notamment obtenir des informations sur les règles applicables, analyser certains documents et consulter un indicateur de qualité de vie lié à la localisation du bien.

1.4.4 Positionnement de PropIQ

L’étude des solutions existantes montre qu’il existe plusieurs outils performants dans leurs domaines respectifs, mais qu’ils répondent à des besoins différents.

Les solutions juridiques basées sur l’IA sont principalement orientées vers la recherche juridique générale. Les plateformes immobilières, quant à elles, sont principalement destinées à la recherche et à la consultation de biens.

PropIQ se situe entre ces deux approches. La solution combine plusieurs fonctionnalités dans une même application mobile :

- une recherche juridique basée sur le RAG ;
- une analyse des règles applicables au contexte immobilier de Setúbal ;
- une possibilité d'analyse de documents immobiliers ;
- une interaction avec l'utilisateur sous forme de chatbot ;
- des réponses accompagnées de références issues du corpus utilisé ;
- un indicateur de qualité de vie basé sur des données géographiques et environnementales ;
- une approche destinée notamment aux acheteurs et investisseurs étrangers.

Cette combinaison constitue l'un des principaux éléments de différenciation de PropIQ. L'objectif n'est pas de remplacer un professionnel du droit ou de l'immobilier, mais de fournir à l'utilisateur une première analyse structurée qui facilite la compréhension d'un bien avant une décision ou une consultation professionnelle.

1.5 Méthodologie CRISP-DM

Pour organiser les différentes étapes du projet, la méthodologie **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) a été adoptée.

CRISP-DM propose une démarche structurée pour les projets utilisant les données et l'intelligence artificielle. Elle permet de partir du besoin métier, de comprendre et préparer les données, de développer les modèles, puis d'évaluer et de déployer la solution. La méthodologie comprend six phases : compréhension du métier, compréhension des données, préparation des données, modélisation, évaluation et déploiement [1].

Le modèle CRISP-DM n'impose toutefois pas une progression strictement linéaire. Les différentes phases peuvent être revisitées lorsque les résultats obtenus nécessitent une modification des données, des modèles ou des objectifs du projet [1].

Dans le cas de PropIQ, cette méthodologie est particulièrement adaptée puisque le projet combine plusieurs types de données et plusieurs composants d'intelligence artificielle. Le système utilise notamment un corpus juridique au format PDF, des données

géographiques et environnementales, des modèles d'embeddings, un système RAG et des modèles de langage.

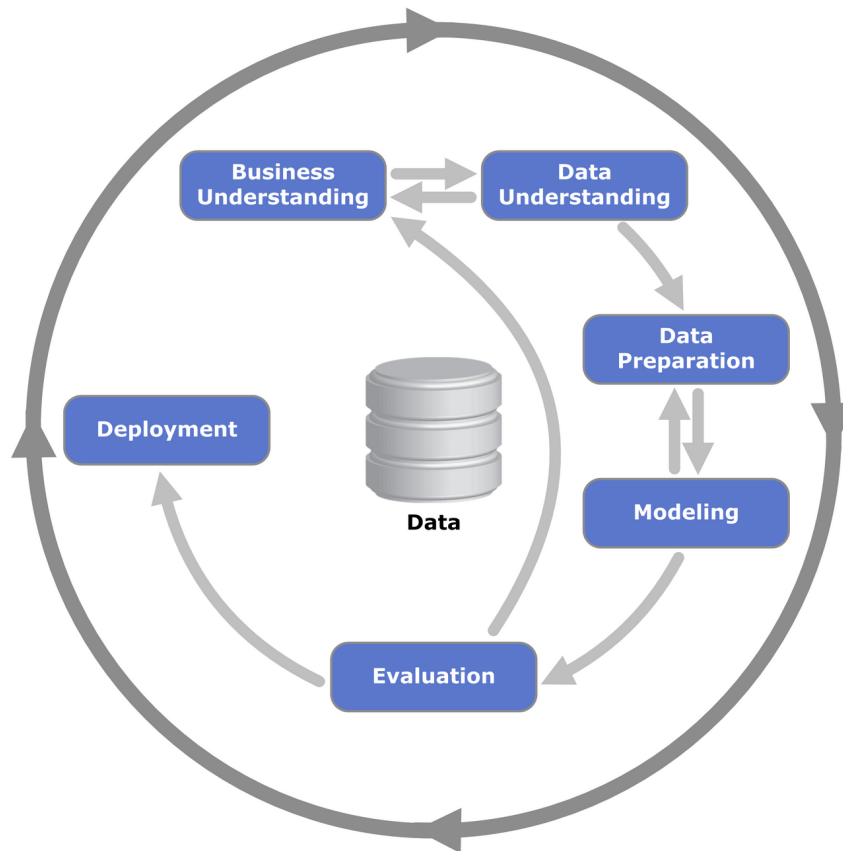


Figure 1.2: Les différentes phases de la méthodologie CRISP-DM [1]

Les principales phases de CRISP-DM sont adaptées au déroulement du projet PropIQ comme suit :

1. **Compréhension du métier :** identification du besoin lié à l'analyse immobilière et à la vérification réglementaire au Portugal, avec un intérêt particulier pour Setúbal.
2. **Compréhension des données :** identification et étude des documents juridiques, des données géographiques et des données utilisées pour l'évaluation de la qualité de vie.
3. **Préparation des données :** collecte, nettoyage, structuration et découpage des documents réglementaires afin de les rendre exploitables par le système RAG.
4. **Modélisation :** mise en place des embeddings, de la recherche sémantique et du système RAG, ainsi que l'intégration des modèles de langage.

5. **Évaluation** : vérification de la qualité des documents récupérés et évaluation des réponses générées par les différents modèles utilisés dans PropIQ.
6. **Déploiement** : intégration des différents composants dans l’application mobile et mise à disposition des fonctionnalités à travers l’architecture de la solution.

Le choix de CRISP-DM permet ainsi de garder une progression claire tout au long du projet. Les différentes phases sont liées entre elles et certaines peuvent être répétées lorsque les résultats obtenus nécessitent une amélioration des données, des modèles ou des paramètres du système [1].

Cette organisation correspond bien au fonctionnement de PropIQ, puisque la qualité d’une réponse juridique dépend à la fois de la qualité du corpus, de la recherche sémantique, du modèle de langage et de l’évaluation des résultats.

1.6 Conclusion

Dans ce chapitre, le contexte général du projet PropIQ a été présenté. La présentation de l’organisme d’accueil a permis de situer le projet dans le domaine immobilier portugais.

Le cadre réglementaire a ensuite été étudié en mettant en évidence la complémentarité entre les règles nationales, le programme Simplex et les documents d’urbanisme applicables à Setúbal. Cette diversité de sources constitue l’une des principales difficultés rencontrées lors de l’analyse réglementaire d’un bien immobilier.

L’étude des solutions existantes a également permis de mettre en évidence les limites des outils étudiés. Les solutions juridiques basées sur l’IA proposent des fonctions de recherche avancées mais restent généralistes, tandis que les plateformes immobilières se concentrent principalement sur la recherche de biens. Aucune des solutions étudiées ne réunit, dans une même application adaptée au contexte de Setúbal, la recherche juridique par RAG, l’analyse de documents immobiliers et l’évaluation de la qualité de vie.

PropIQ a ainsi été positionné comme une solution d’aide à la décision destinée à compléter les outils immobiliers existants. Son objectif est de faciliter l’accès à des infor-

mations réglementaires et contextuelles utiles aux acheteurs et investisseurs, tout en conservant une approche basée sur des sources vérifiables.

Enfin, la méthodologie CRISP-DM a été retenue pour organiser le développement du projet. Elle servira de fil conducteur pour les étapes suivantes, depuis l’étude du besoin et des données jusqu’à la conception, l’évaluation et le déploiement de PropIQ [1].

Le chapitre suivant présente plus en détail la compréhension du besoin métier ainsi que les données utilisées dans le cadre du projet.

CHAPITRE 2

Compréhension des métiers et des données

2.1 Introduction

Ce chapitre correspond aux deux premières phases de la méthodologie CRISP-DM présentée dans le chapitre précédent : la **compréhension du métier** (*Business Understanding*) et la **compréhension des données** (*Data Understanding*) [1].

La première phase permet de mieux définir la problématique à laquelle PropIQ doit répondre. Elle consiste à identifier les besoins des utilisateurs, les acteurs concernés ainsi que les objectifs du projet. Ces éléments sont ensuite traduits en objectifs techniques afin de guider la conception de la solution.

La deuxième phase porte sur les données utilisées par PropIQ. Elle présente les différentes sources exploitées par le système, notamment les documents juridiques utilisés par le chatbot RAG ainsi que les données géographiques et environnementales utilisées pour l'évaluation de la qualité de vie.

L'objectif de ce chapitre est donc de faire le lien entre le besoin métier, les fonctionnalités attendues et les données nécessaires au fonctionnement de la solution.

2.2 Compréhension du métier

2.2.1 Contexte métier et problématique

Comme présenté dans le chapitre précédent, l’analyse d’un bien immobilier au Portugal peut nécessiter la consultation de plusieurs règles et documents. Le programme Simplex constitue une première composante du cadre réglementaire national, mais il est complété par des règles locales propres à chaque commune, notamment celles de Setúbal [2, 4, 5].

Cette situation rend la vérification de la conformité d’un bien relativement complexe. Pour obtenir une information fiable, l’utilisateur doit parfois consulter plusieurs documents et comprendre des textes juridiques qui ne sont pas toujours faciles à interpréter pour une personne non spécialisée.

Cette problématique concerne directement Remax Wise (Portugal), dont l’activité consiste notamment à accompagner des clients dans leurs projets immobiliers. Elle concerne également les particuliers et les investisseurs, en particulier les clients internationaux qui peuvent avoir des difficultés à comprendre la réglementation locale.

PropIQ a donc été conçu pour faciliter cette première étape d’analyse. L’objectif est de permettre à l’utilisateur de poser une question en langage naturel et d’obtenir une réponse basée sur les documents réglementaires intégrés dans le système. La solution ajoute également des informations sur l’environnement du bien afin de fournir une vision plus globale de la situation.

La recherche augmentée par génération (*Retrieval-Augmented Generation*, RAG) permet précisément de combiner la recherche d’informations pertinentes dans une base documentaire avec la génération d’une réponse par un modèle de langage [13]. Cette approche est particulièrement adaptée au contexte de PropIQ, puisque les réponses doivent être liées à un corpus réglementaire défini.

2.2.2 Objectifs métier et objectifs Data Science

Les besoins identifiés précédemment sont traduits en objectifs techniques afin de guider le développement de PropIQ. Cette correspondance permet de vérifier que les fonctionnalités développées répondent réellement aux objectifs du projet.

Le tableau 2.1 présente cette relation entre les objectifs métier et les objectifs liés à la

Data Science et à l'intelligence artificielle.

Table 2.1: Correspondance entre les objectifs métier et les objectifs Data Science

Objectif métier	Objectif Data Science / IA
Réduire le temps nécessaire pour vérifier la conformité d'un bien	Automatiser la recherche d'informations réglementaires à l'aide d'un pipeline RAG basé sur le corpus juridique de PropIQ [13]
Faciliter la compréhension des règles pour les non-spécialistes	Générer des réponses en langage naturel et fournir les références des textes utilisés
Évaluer un bien au-delà de sa situation juridique	Calculer un indice de qualité de vie de 0 à 100 à partir de données géographiques et environnementales
Faciliter l'interaction avec le système	Permettre l'utilisation de plusieurs types d'entrée : texte, image, document et voix

Cette correspondance permet de définir une direction claire pour le développement. Chaque objectif métier est ainsi associé à une fonctionnalité ou à un composant technique de PropIQ.

2.2.3 Identification des acteurs

Plusieurs acteurs interviennent directement ou indirectement dans l'utilisation et le fonctionnement de PropIQ :

- **Utilisateur particulier** : utilise PropIQ pour obtenir des informations sur un bien immobilier avant de prendre une décision d'achat ou d'investissement.
- **Agent immobilier** : peut utiliser la solution comme outil d'aide à l'analyse et à l'accompagnement de ses clients.
- **Administrateur** : assure la gestion des utilisateurs, des quotas ainsi que de la base de connaissances utilisée par le système.

- **Services externes** : regroupent les différents services nécessaires au fonctionnement de l'application, notamment les fournisseurs de modèles de langage via OpenRouter, Firebase pour l'authentification et les services de paiement.

L'utilisation d'OpenRouter permet notamment d'accéder à plusieurs modèles de langage à travers une interface API unifiée [14]. Firebase Authentication fournit quant à lui des mécanismes d'authentification prenant en charge notamment l'inscription par adresse e-mail et mot de passe ainsi que les fournisseurs d'identité fédérés tels que Google [15].

Ces différents acteurs permettent de définir les principales interactions avec la solution et servent de base à l'identification des fonctionnalités nécessaires.

2.2.4 Besoins fonctionnels

Les besoins fonctionnels correspondent aux services que PropIQ doit fournir à ses utilisateurs. Ils ont été regroupés dans un backlog et classés selon leur niveau de priorité à l'aide de la méthode MoSCoW : *Must have*, *Should have* et *Could have*. Cette technique de priorisation permet de distinguer les fonctionnalités indispensables des fonctionnalités importantes ou optionnelles [16].

Le tableau 2.2 présente les principaux besoins fonctionnels identifiés.

Table 2.2: Besoins fonctionnels de PropIQ

ID	Besoin	Description	Priorité
BF01	Chatbot juridique	Comprendre la question de l'utilisateur et rechercher les informations pertinentes dans le corpus juridique à l'aide du RAG	Must have
BF02	Décision synthétique	Présenter une conclusion simple indiquant si l'opération est possible, possible sous certaines conditions ou non conforme	Must have
BF03	Justification des réponses	Fournir les références des articles, décrets et règlements utilisés pour construire la réponse	Must have
BF04	Analyse de la zone	Collecter et analyser des informations publiques concernant l'environnement du bien	Should have
BF05	Indice de qualité de vie	Calculer un score de 0 à 100 à partir de plusieurs catégories comme les transports, l'éducation, la santé, les commerces et la qualité de l'air	Should have
BF06	Gestion des comptes	Permettre l'inscription, l'authentification et la gestion des offres d'utilisation	Must have
BF07	Historique et export	Conserver les analyses précédentes et permettre leur consultation ou leur exportation	Could have

Parmi ces besoins, le chatbot juridique constitue une fonctionnalité centrale de PropIQ. Il permet d'interroger le corpus réglementaire en langage naturel et de fournir une réponse accompagnée des sources utilisées. Le module de qualité de vie complète cette

fonctionnalité en apportant des informations sur l’environnement du bien.

2.2.5 Besoins non fonctionnels

En plus des fonctionnalités attendues, PropIQ doit respecter plusieurs exigences liées à la qualité globale du système. Ces exigences concernent notamment les performances, la sécurité, la fiabilité et la possibilité de faire évoluer la solution.

Le tableau 2.3 présente les principaux besoins non fonctionnels retenus.

Table 2.3: Besoins non fonctionnels de PropIQ

ID	Besoin	Description	Priorité
BNF01	Performance	Le chatbot doit fournir une réponse dans un délai compatible avec une utilisation interactive	Must have
BNF02	Fiabilité	Les réponses juridiques doivent être basées sur des sources identifiables et le système doit limiter autant que possible les hallucinations des modèles de langage	Must have
BNF03	Sécurité	Les comptes et les données des utilisateurs doivent être protégés grâce à une authentification adaptée et à des mécanismes de sécurité appropriés	Must have
BNF04	Scalabilité	L'architecture doit pouvoir évoluer lorsque le nombre d'utilisateurs et de requêtes augmente	Should have
BNF05	Maintenabilité	La structure du système doit faciliter la mise à jour du corpus et des différents composants de la solution	Should have
BNF06	Observabilité	Les performances des modèles et du système doivent pouvoir être suivies à l'aide de métriques adaptées	Should have
BNF07	Portabilité	La solution doit être accessible sur les plateformes mobiles ciblées par le projet	Could have

Ces exigences sont particulièrement importantes pour un système comme PropIQ. La qualité d'une réponse ne dépend pas uniquement du modèle d'intelligence artificielle utilisé, mais également de la disponibilité des sources, de la stabilité du système et de la sécurité des données.

Dans le cas de l'authentification, Firebase Authentication fournit notamment des mécanismes permettant de gérer l'identité des utilisateurs et de vérifier les informations

d'authentification côté backend [15].

2.2.6 Product Backlog

Le product backlog regroupe les principales fonctionnalités à développer pour répondre aux besoins identifiés. Les fonctionnalités sont exprimées sous forme de *User Stories*, ce qui permet de les présenter du point de vue de l'utilisateur ou de l'administrateur.

Le tableau 2.4 présente une synthèse du backlog défini pour PropIQ.

Table 2.4: Product Backlog de PropIQ

ID	Epic	User Story	Priorité
US-01	Authentification	En tant qu'utilisateur, je veux créer un compte et me connecter avec mon adresse e-mail ou mon compte Google afin d'accéder à mes analyses de manière sécurisée	Haute
US-02	Chatbot juridique	En tant qu'utilisateur, je veux poser une question juridique en langage naturel et obtenir une réponse basée sur les textes réglementaires applicables	Haute
US-03	Interaction vocale	En tant qu'utilisateur, je veux poser une question à l'aide de ma voix et écouter la réponse afin de rendre l'interaction plus naturelle	Moyenne
US-04	Analyse de documents	En tant qu'utilisateur, je veux envoyer une image ou un document PDF afin que PropIQ puisse analyser son contenu et répondre à mes questions	Moyenne
US-05	Qualité de vie	En tant qu'utilisateur, je veux renseigner l'adresse d'un bien afin d'obtenir un score basé sur les services et les points d'intérêt situés à proximité	Haute
US-06	Historique	En tant qu'utilisateur, je veux consulter mes analyses précédentes afin de pouvoir retrouver les informations déjà obtenues	Basse
US-07	Abonnement et quotas	En tant qu'utilisateur, je veux accéder à une offre premium afin de bénéficier d'un niveau d'utilisation supérieur aux limites de l'offre gratuite	Moyenne
US-08	Suivi et supervision	En tant qu'administrateur, je veux suivre les performances des modèles et identifier les éventuels problèmes afin de maintenir la fiabilité du système	Haute

Le backlog permet ainsi de relier les besoins identifiés à des fonctionnalités concrètes.

Il constitue également une base pour organiser les différentes étapes de développement et suivre l'évolution du projet.

2.2.7 Critères de succès du projet

Les critères de succès permettent de vérifier si les objectifs définis au début du projet sont effectivement atteints. Dans le cadre de PropIQ, les principaux critères retenus sont les suivants :

- **Pertinence des réponses juridiques** : les réponses générées doivent être cohérentes avec la question posée et les informations présentes dans les documents récupérés.
- **Traçabilité** : les réponses du chatbot doivent pouvoir être reliées aux sources réglementaires utilisées par le système. Les métriques d'évaluation détaillées seront présentées dans le chapitre consacré à l'évaluation.
- **Cohérence de l'indice de qualité de vie** : le score calculé doit être cohérent avec les données géographiques et environnementales disponibles pour la zone étudiée.
- **Qualité du système** : la solution doit respecter les exigences définies en matière de sécurité, de performance, de maintenabilité et d'évolutivité.

Ces critères serviront de référence lors des phases d'évaluation et de validation de PropIQ.

2.3 Compréhension des données

2.3.1 Sources de données

PropIQ utilise plusieurs sources de données nécessaires à ses différentes fonctionnalités. Elles peuvent être regroupées en deux grandes familles : les données réglementaires utilisées par le chatbot juridique et les données géographiques et environnementales utilisées pour le calcul de la qualité de vie.

Le tableau 2.5 présente les principales sources utilisées dans le projet.

Table 2.5: Sources de données utilisées par PropIQ

Composant	Source	Nature des données
Chatbot juridique	Programme Simplex, lois, décrets-lois, règlements municipaux et documents d’urbanisme de Setúbal	Documents juridiques principalement disponibles au format PDF ou HTML, organisés en articles et sections
Qualité de vie	API Overpass / Open-StreetMap	Points d’intérêt géolocalisés : transports, établissements scolaires, structures de santé, commerces et espaces verts [17]
Qualité de vie	Open-Meteo Air Quality API	Données relatives à la qualité de l’air dans différentes zones géographiques [18]

Chaque source répond donc à un besoin spécifique. Les documents réglementaires servent principalement à alimenter le système RAG, tandis que les données géographiques et environnementales sont utilisées pour construire l’indicateur de qualité de vie.

2.3.2 Données juridiques

Le corpus juridique utilisé par PropIQ reprend les différentes catégories de documents présentées dans le chapitre précédent. Il comprend notamment les mesures liées au programme Simplex, les textes législatifs nationaux ainsi que les règlements municipaux et les documents d’urbanisme relatifs à Setúbal [2, 4, 5].

Le corpus collecté contient environ 340 documents répartis en 19 catégories thématiques. Les documents couvrent une période allant de 2010 à 2024.

La majorité des documents sont rédigés en portugais, qui est la langue officielle des publications du *Diário da República*. Afin de faciliter l’utilisation de PropIQ par des utilisateurs internationaux, le système permet de produire les réponses dans différentes

langues, notamment en français et en anglais.

La majorité des documents sont disponibles au format PDF avec du texte sélectionnable. Cependant, certains documents municipaux plus anciens sont disponibles uniquement sous forme de documents scannés. Ces fichiers nécessitent alors une étape supplémentaire de traitement, notamment l'utilisation de l'OCR, afin d'extraire leur contenu textuel.

Cette diversité des formats doit être prise en compte lors de la préparation du corpus, car la qualité du texte extrait influence directement les performances de la recherche sémantique et du système RAG.

2.3.3 Données géographiques et environnementales

Le module de qualité de vie de PropIQ utilise des données collectées dynamiquement à partir de l'adresse du bien immobilier analysé.

Après géocodage de l'adresse, le système interroge différentes sources afin d'identifier les services et les éléments présents autour du bien. Le géocodage permet de transformer une adresse en coordonnées géographiques, notamment en latitude et longitude, qui peuvent ensuite être utilisées pour effectuer des recherches géographiques [19].

L'API Overpass, basée sur les données d'OpenStreetMap, permet notamment de récupérer les éléments géographiques répondant à des critères définis par l'application. Contrairement à l'API principale d'OpenStreetMap destinée notamment à l'édition, Overpass est conçue pour les consommateurs de données souhaitant interroger une partie spécifique des données OSM selon des critères tels que la localisation, le type d'objet ou les tags [17].

Les points d'intérêt sont regroupés en plusieurs catégories :

- **Transports** : arrêts de bus, gares et autres moyens de transport disponibles à proximité ;
- **Éducation** : écoles primaires, collèges et lycées ;
- **Santé** : hôpitaux, centres de santé et pharmacies ;
- **Commerces** : supermarchés et commerces de proximité ;

- **Espaces verts** : parcs, jardins et autres espaces publics similaires.

En complément de ces données, l'**Open-Meteo Air Quality API** est utilisée pour récupérer des informations liées à la qualité de l'air. Cette API permet d'interroger, à partir de coordonnées géographiques, les concentrations actuelles de particules fines $PM_{2.5}$ et PM_{10} ainsi que l'indice de qualité de l'air américain (US AQI) [18].

Ces différentes informations sont ensuite utilisées par le module de qualité de vie pour produire un indicateur synthétique permettant à l'utilisateur d'avoir une première vision de l'environnement du bien.

2.3.4 Exploration et qualité des données

L'exploration initiale des données a permis d'identifier plusieurs difficultés liées à la nature des sources utilisées.

Concernant les documents juridiques, les principales difficultés observées sont les suivantes :

- **Formats et structures différents** : les documents ne présentent pas tous la même organisation selon leur origine et leur niveau réglementaire ;
- **Références croisées** : certains textes font référence à d'autres lois ou règlements, ce qui doit être pris en compte lors de la préparation des documents ;
- **Versions multiples** : certains textes ont été modifiés plusieurs fois. Il est donc nécessaire d'identifier la version pertinente afin d'éviter l'utilisation simultanée de contenus contradictoires ;
- **Documents anciens** : certains textes plus anciens présentent une structure moins régulière, ce qui peut rendre leur découpage automatique plus difficile.

Ces problèmes ont une influence directe sur la qualité du système RAG. Un document mal extrait ou mal découpé peut produire des résultats moins pertinents lors de la recherche sémantique. La préparation des données constitue donc une étape importante du projet.

Les données géographiques présentent également certaines limites. La quantité de points d'intérêt disponibles sur OpenStreetMap n'est pas identique dans toutes les zones. Les secteurs urbains disposent généralement d'une couverture plus importante que certaines zones périphériques.

Cette différence peut avoir un impact sur le calcul de l'indice de qualité de vie. Un faible nombre de points d'intérêt dans une zone ne signifie pas nécessairement que les services sont absents ; il peut également correspondre à une couverture moins complète des données disponibles.

Concernant Open-Meteo, les valeurs de qualité de l'air reposent sur un modèle de prévision atmosphérique (CAMS Europe) plutôt que sur des relevés directs de stations locales. La précision de ces estimations peut donc varier selon la zone géographique considérée [18]. Cette limitation doit donc être prise en compte lors de l'interprétation du score environnemental.

Pour vérifier la cohérence des résultats, plusieurs adresses de test situées dans le centre de Setúbal et dans des zones périphériques ont été utilisées. Les points d'intérêt obtenus ont été comparés avec une carte de référence. Cette vérification a permis de confirmer globalement la cohérence des résultats tout en mettant en évidence une représentation parfois limitée de certaines catégories, notamment les espaces verts dans les zones moins couvertes par OpenStreetMap.

Ces observations seront prises en compte lors de la préparation et du traitement des données présentés dans le chapitre suivant.

2.4 Conclusion

Ce chapitre a permis de définir le besoin métier et de le traduire en objectifs techniques pour le projet PropIQ. Les principaux acteurs, les besoins fonctionnels et non fonctionnels ainsi que le product backlog ont été présentés afin de préciser les fonctionnalités attendues de la solution.

La deuxième partie du chapitre a porté sur la compréhension des données. Les principales sources utilisées par PropIQ ont été identifiées : les documents réglementaires pour le chatbot juridique et les données géographiques et environnementales pour le

module de qualité de vie.

L’exploration de ces données a également permis d’identifier plusieurs difficultés, notamment la diversité des formats des documents juridiques, les différentes versions des textes, les références croisées ainsi que les différences de couverture des données géographiques et environnementales.

Ces observations montrent que la préparation des données constitue une étape essentielle pour obtenir des résultats fiables. Le chapitre suivant sera donc consacré à la préparation et au traitement des données utilisées par PropIQ, notamment la préparation du corpus juridique destiné au pipeline RAG et le traitement des données nécessaires au calcul de l’indice de qualité de vie.

CHAPITRE 3

Préparation des données

3.1 Introduction

Ce chapitre présente la phase de *Data Preparation* de la méthodologie CRISP-DM. Cette étape est importante pour PropIQ, car la qualité des données utilisées influence directement les résultats produits par le système. Dans CRISP-DM, la préparation des données consiste notamment à nettoyer, transformer et organiser les données afin de les rendre adaptées aux étapes de modélisation et d'analyse [1, 20].

Deux familles de données sont préparées dans le cadre du projet. La première concerne les données juridiques utilisées par le chatbot basé sur le RAG. Elle comprend la collecte des textes, leur nettoyage, leur organisation, leur segmentation et leur transformation en vecteurs.

La seconde famille correspond aux données géographiques et environnementales utilisées par le module de qualité de vie. Ces données sont obtenues à partir de l'adresse du bien et doivent être nettoyées et organisées avant leur utilisation dans le calcul du score.

L'objectif de cette phase est donc de transformer les données brutes en données structurées et exploitables par les différents composants de PropIQ.

3.2 Préparation des données juridiques

3.2.1 Collecte des textes depuis le Diário da República

Les documents juridiques utilisés par PropIQ sont principalement issus du portail officiel Diário da República Eletrónico [21], qui constitue la plateforme officielle de consultation des actes publiés au Diário da República.

La collecte concerne notamment les textes liés au programme Simplex, les lois et décrets-lois nationaux ainsi que les règlements municipaux utiles à l'analyse immobilière.

Afin de faciliter l'organisation du corpus et la recherche ultérieure, les documents ont été répartis en 19 catégories thématiques. Cette classification permet de regrouper les textes selon leur domaine et de faciliter leur utilisation dans le système RAG.

Les tableaux 3.1 et 3.2 présentent les catégories retenues.

Table 3.1: Catégories thématiques du corpus juridique (1/2)

Code	Catégorie / Domaine couvert
ACCESSIBILITY	Lois et normes d'accessibilité pour les bâtiments publics et privés
CIVIL_SALE	Garanties civiles liées à la vente et à la construction immobilière
CIVIL_FINANCIAL	Obligations financières civiles relatives à la propriété immobilière
CIVIL_REGULATORY	Services et réglementation immobilière civile
CONDOMINIUM	Règles relatives aux copropriétés et à leur gestion
ENERGY	Réglementation relative à la performance et à l'efficacité énergétique des bâtiments
FIRE_SAFETY	Réglementation relative à la sécurité incendie des bâtiments
FOREIGN_STATUS	Réglementation concernant les étrangers et les investissements immobiliers
HOUSING	Réglementation générale relative au logement

Table 3.2: Catégories thématiques du corpus juridique (2/2)

Code	Catégorie / Domaine couvert
LOCAL_ACCOMMODATION	Réglementation relative à l'hébergement local et touristique
MORTGAGE	Réglementation relative aux crédits et garanties hypothécaires
PDM_SETUBAL	Plan Directeur Municipal et règles d'urbanisme de Setúbal
RAN	Régime de la Reserva Agrícola Nacional
REN	Régime de la Reserva Ecológica Nacional
RGEU	Régime général des édifications urbaines
RJUE	Régime juridique de l'urbanisation et de l'édification
SIMPLEX	Mesures et dispositions associées au programme Simplex
URBAN_ZONING	Règles d'urbanisme et de zonage applicables aux biens immobiliers

La collecte des documents a été réalisée manuellement à partir des versions disponibles sur le portail officiel. Ce choix permet de sélectionner directement les documents considérés comme pertinents pour le corpus et d'éviter l'intégration automatique de versions non souhaitées.

Le corpus obtenu contient environ 340 documents répartis entre les 19 catégories. La répartition n'est pas uniforme : certaines catégories, comme URBAN_ZONING, RGEU et CIVIL_SALE, contiennent davantage de documents en raison de leur importance pour l'analyse immobilière. D'autres catégories plus spécifiques contiennent un nombre plus limité de documents.

La période couverte par les documents s'étend principalement de 2010 à 2024. Une attention particulière est accordée aux textes les plus récents lorsque les réglementations concernées sont susceptibles d'avoir été modifiées.

L'utilisation de sources officielles constitue un élément important pour PropIQ, puisque

les réponses produites par le chatbot doivent être basées sur des documents réglementaires identifiables.

3.2.2 Nettoyage et prétraitement textuel

Les documents PDF collectés ne peuvent pas être utilisés directement par le pipeline RAG. Une étape de nettoyage est donc nécessaire afin de supprimer les éléments qui peuvent perturber la recherche sémantique.

Les principales opérations réalisées sont les suivantes :

- **Suppression des éléments non pertinents** : suppression de certains en-têtes, pieds de page, numéros de page et autres éléments ne contenant pas d'information juridique utile ;
- **Normalisation des caractères** : normalisation des caractères et de l'encodage, notamment pour les caractères spécifiques à la langue portugaise ;
- **Suppression des doublons** : élimination des versions répétées d'un même document ;
- **Correction de la mise en page** : correction des problèmes provoqués par l'extraction des fichiers PDF, tels que les retours à la ligne incorrects ou la séparation des colonnes.

L'extraction du contenu textuel est réalisée avec la bibliothèque Python *pdfplumber*. Cette bibliothèque permet notamment d'extraire le texte des pages PDF et de préserver autant que possible la structure du contenu extrait [22].

Après l'extraction, des expressions régulières sont utilisées pour identifier certains éléments répétitifs présents dans les documents du Diário da República. Cette étape permet notamment de supprimer les informations de pagination et certains éléments d'en-tête qui ne sont pas utiles pour la recherche.

Le texte obtenu après ces traitements constitue la base utilisée pour l'étape suivante : la segmentation des documents.

3.2.3 Segmentation des textes (chunking)

Après le nettoyage, les documents sont découpés en petites unités de texte appelées *chunks*. Cette étape est nécessaire pour permettre au système RAG de retrouver plus facilement les passages correspondant à la question de l'utilisateur.

Pour les documents juridiques, un découpage uniquement basé sur un nombre fixe de caractères peut séparer des informations qui doivent rester ensemble. La stratégie retenue dans PropIQ privilégie donc la structure naturelle des textes juridiques.

Chaque article est d'abord considéré comme une unité de base. Lorsqu'un article reste suffisamment court, il est conservé sous la forme d'un seul chunk. Lorsqu'il dépasse la longueur maximale définie, il est divisé en plusieurs parties.

Dans l'implémentation retenue, la longueur maximale du chunk est fixée à 2000 caractères, avec un overlap de 200 caractères entre deux chunks successifs.

L'utilisation des caractères plutôt que des tokens est ici cohérente avec le traitement appliqué avant l'encodage. Elle permet également d'éviter de confondre la longueur du texte découpé avec la limite de contexte du modèle d'embeddings.

L'overlap permet de conserver une partie du contexte entre deux fragments consécutifs. Ainsi, lorsqu'une information importante se trouve à la limite entre deux morceaux, le contexte n'est pas complètement perdu.

Cette stratégie permet de conserver un meilleur équilibre entre deux besoins : disposer de passages suffisamment courts pour une recherche précise et conserver suffisamment de contexte pour interpréter correctement une disposition juridique.

3.2.4 Génération des embeddings vectoriels

Une fois les documents segmentés, chaque chunk est transformé en représentation vectorielle à l'aide du modèle d'embeddings multilingue *multilingual-e5-base* [23].

Le choix de ce modèle est lié au caractère multilingue de PropIQ. Le modèle *multilingual-e5-base* est basé sur XLM-RoBERTa et prend en charge de nombreuses langues, ce qui le rend adapté à la représentation sémantique de textes multilingues [24].

Les embeddings permettent de représenter les textes sous forme de vecteurs numériques.

Deux contenus ayant un sens proche peuvent ainsi être rapprochés dans l'espace vectoriel, même si leur formulation exacte est différente.

Le modèle produit des représentations de dimension 768. Cette caractéristique est définie dans la configuration du modèle *multilingual-e5-base* [25].

Cette représentation constitue une étape essentielle du RAG, car elle permet ensuite de comparer la question de l'utilisateur avec les différents passages du corpus.

3.2.5 Indexation dans la base vectorielle

Les vecteurs générés sont ensuite stockés dans la base vectorielle Pinecone. Cette base permet de stocker les représentations vectorielles et de les interroger afin d'identifier les passages les plus proches d'une requête donnée [26].

Chaque vecteur est enregistré avec plusieurs métadonnées associées au document d'origine. Ces informations permettent notamment de retrouver la référence du texte, sa catégorie, son niveau réglementaire et, lorsque cela est disponible, la zone géographique concernée. Pinecone permet notamment d'associer des métadonnées aux vecteurs lors de leur insertion dans l'index [?].

Ces métadonnées jouent un rôle important dans PropIQ. Elles permettent de conserver le lien entre le passage récupéré et le document original, ce qui facilite ensuite la présentation des références utilisées dans la réponse.

Le modèle *multilingual-e5-base* produit des vecteurs de dimension 768. La similarité entre les vecteurs est calculée à l'aide de la similarité cosinus, adaptée à la comparaison des représentations vectorielles utilisées dans le système.

L'index peut également être mis à jour progressivement. Lorsqu'un nouveau document réglementaire doit être ajouté, ses chunks peuvent être transformés en vecteurs puis insérés dans l'index sans devoir reconstruire l'ensemble de la base. Cette possibilité facilite la maintenance du corpus juridique.

3.3 Préparation des données géographiques et environnementales

3.3.1 Géocodage des adresses

Pour calculer la qualité de vie autour d'un bien, PropIQ doit d'abord transformer l'adresse saisie par l'utilisateur en coordonnées géographiques.

Cette opération est réalisée à l'aide du service de géocodage Nominatim, basé sur les données d'OpenStreetMap. Nominatim fournit notamment une API permettant de rechercher des objets à partir d'un nom ou d'une adresse et de retourner leurs informations géographiques, notamment la latitude et la longitude [27, 28].

Ces coordonnées sont ensuite utilisées pour rechercher les points d'intérêt situés autour du bien et pour associer les données environnementales à la zone concernée.

Lorsque l'adresse fournie n'est pas suffisamment précise ou peut correspondre à plusieurs résultats, plusieurs correspondances peuvent être retournées. Dans ce cas, l'utilisateur est invité à préciser son adresse afin d'obtenir une localisation plus fiable.

L'utilisation de l'instance publique de Nominatim doit par ailleurs respecter les règles d'utilisation définies par l'OpenStreetMap Foundation, notamment en matière de fréquence des requêtes et d'identification de l'application [29].

3.3.2 Collecte et normalisation des points d'intérêt

Une fois les coordonnées obtenues, PropIQ interroge l'API Overpass afin de récupérer les points d'intérêt disponibles dans un rayon de 3 km autour du bien.

L'API Overpass est une API en lecture seule conçue pour interroger des parties sélectionnées des données OpenStreetMap selon différents critères, notamment la localisation, le type d'objet, les tags et la proximité [17].

Les résultats sont regroupés en cinq grandes catégories utilisées pour le calcul de l'indice de qualité de vie :

- **Transports** : arrêts de bus, gares et autres infrastructures de transport ;
- **Éducation** : écoles et autres établissements scolaires ;
- **Santé** : hôpitaux, centres de santé et pharmacies ;
- **Commerces** : supermarchés et commerces de proximité ;

- **Espaces verts** : parcs, jardins et autres espaces verts.

Les données retournées par OpenStreetMap peuvent présenter des différences de structure et de classification. Une étape de normalisation est donc appliquée avant leur utilisation.

Les doublons sont notamment regroupés lorsqu'ils correspondent au même point d'intérêt ou à des coordonnées très proches. Les entrées qui ne permettent pas d'identifier correctement une catégorie utile au calcul du score sont également écartées.

Cette préparation permet d'obtenir un ensemble de données plus homogène pour les étapes suivantes du calcul de l'indice de qualité de vie.

3.3.3 Normalisation des données de qualité de l'air

En complément des points d'intérêt, PropIQ utilise des données relatives à la qualité de l'air provenant de l'**Open-Meteo Air Quality API**.

Cette API permet d'interroger des données de qualité de l'air à partir de coordonnées géographiques, et fournit notamment des mesures associées à différents polluants atmosphériques [18].

Ces données sont associées à la zone du bien afin d'intégrer la dimension environnementale dans l'indice de qualité de vie. Les principaux indicateurs considérés sont les particules PM2.5, les particules PM10 et le dioxyde d'azote NO2, lorsqu'ils sont disponibles.

Lorsque plusieurs points de mesure sont accessibles autour du bien, les mesures peuvent être combinées en tenant compte de leur distance par rapport à la localisation étudiée.

Dans le cas où aucune mesure suffisamment proche ou récente n'est disponible, une valeur de référence est utilisée afin que l'absence de données ne provoque pas directement une pénalisation du score.

Cette étape permet donc d'intégrer les informations environnementales avec les autres données utilisées par le module de qualité de vie.

3.4 Conclusion

Ce chapitre a présenté les principales étapes de préparation des données utilisées dans PropIQ.

Pour le chatbot juridique, les documents provenant du Diário da República ont été collectés, organisés en 19 catégories, nettoyés puis découpés en chunks. Les passages obtenus sont ensuite transformés en embeddings avec *multilingual-e5-base* et indexés dans Pinecone afin d’être utilisés par le système RAG.

Pour le module de qualité de vie, les adresses sont d’abord géocodées à l’aide de Nominatim. Les points d’intérêt sont ensuite récupérés avec Overpass à partir d’OpenStreetMap et regroupés selon plusieurs catégories. Des données relatives à la qualité de l’air sont également récupérées via l’**Open-Meteo Air Quality API**, tandis que les informations concernant les transports, les établissements scolaires, les structures de santé, les commerces et les espaces verts proviennent principalement d’OpenStreetMap et sont interrogées à l’aide de l’API Overpass.

L’ensemble de ces traitements permet de transformer les données brutes en données adaptées aux différents composants de PropIQ. La préparation du corpus juridique constitue notamment une étape essentielle pour améliorer la qualité de la recherche et la traçabilité des réponses du chatbot.

Le chapitre suivant présente la phase de modélisation et d’évaluation, consacrée à la mise en place du pipeline RAG, à l’utilisation des modèles de langage ainsi qu’à l’évaluation des résultats obtenus.

CHAPITRE 4

Modélisation et évaluation

4.1 Introduction

Ce quatrième chapitre présente les phases de *Modeling* et d'*Evaluation* de la méthodologie CRISP-DM appliquées à PropIQ. Il décrit d'abord la conception du pipeline de recherche augmentée par génération (RAG), qui constitue le cœur du chatbot juridique. Les différents modèles d'intelligence artificielle utilisés dans l'application sont ensuite présentés selon leur fonction : traduction, transcription vocale, raisonnement juridique et analyse multimodale.

La seconde partie du chapitre est consacrée à l'évaluation comparative des modèles de langage utilisés pour le raisonnement juridique. Cette évaluation s'appuie sur plusieurs critères, notamment le temps de réponse, la pertinence des réponses et les hallucinations détectées. Le suivi des expérimentations est assuré avec MLflow afin de faciliter la comparaison des résultats et le choix du modèle utilisé par défaut dans PropIQ [30].

4.2 Modélisation

4.2.1 Modélisation du pipeline RAG

Le chatbot juridique de PropIQ repose sur une architecture de *Retrieval-Augmented Generation* (RAG). Cette approche consiste à combiner la recherche d'informations dans une base documentaire avec la génération de réponses par un modèle de langage

[13].

Lorsqu’un utilisateur pose une question, celle-ci est transformée en représentation vectorielle à l’aide du modèle d’embeddings multilingue `multilingual-e5-base`. La représentation obtenue est ensuite comparée aux vecteurs des passages juridiques stockés dans la base vectorielle Pinecone, selon une similarité cosinus calculée sur des embeddings normalisés. Les passages les plus proches sont récupérés à l’aide d’une recherche *top-k*, avec $k = 15$.

Les résultats sont ensuite filtrés selon un seuil de similarité minimal fixé à 0,30 : seuls les passages dont le score dépasse ce seuil sont conservés. Si moins de cinq passages satisfont ce seuil, un mécanisme de repli (*fallback*) est déclenché et retourne directement les cinq meilleurs résultats bruts, afin de garantir qu’un contexte minimal soit toujours transmis au modèle de langage même lorsque la question s’éloigne du corpus indexé. Dans tous les cas, au maximum dix passages sont conservés et assemblés pour former le contexte final, chaque passage étant tronqué à 1 400 caractères et le contexte global limité à 14 000 caractères afin de respecter la fenêtre de contexte des modèles utilisés.

Les passages sélectionnés sont ensuite transmis au modèle de langage avec la question de l’utilisateur. La génération de la réponse est effectuée avec une température nulle (`temperature = 0.0`), afin de privilégier des réponses déterministes et de limiter la variabilité d’une exécution à l’autre — un choix particulièrement important dans un contexte juridique où la stabilité des réponses est prioritaire sur leur créativité — et une longueur maximale de génération de 800 tokens (`max_tokens = 800`).

Cette architecture permet ainsi de limiter la génération de réponses fondées uniquement sur les connaissances internes du modèle et de privilégier les informations provenant du corpus juridique préparé pour PropIQ.

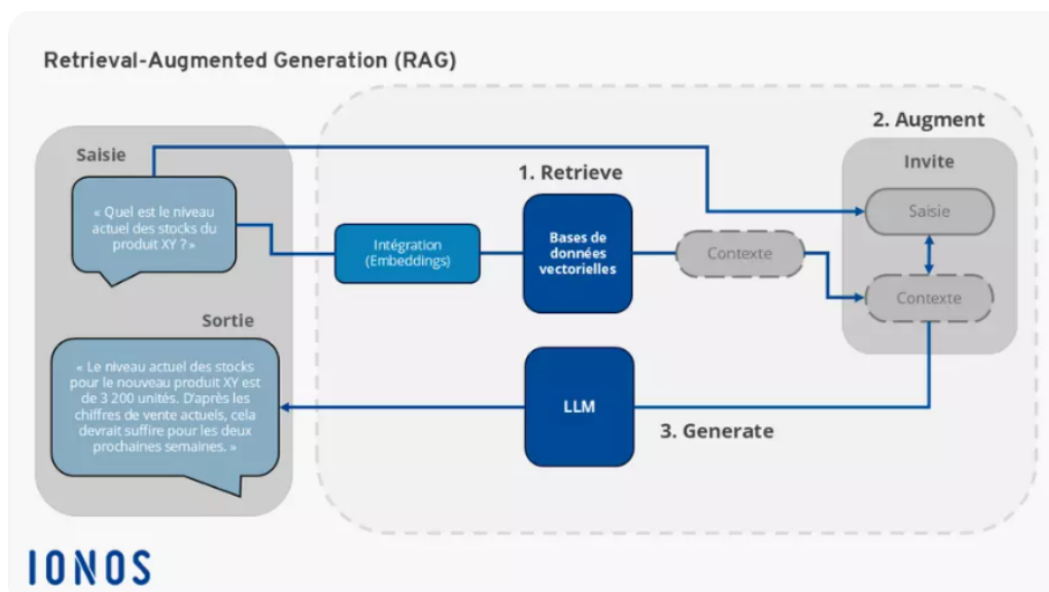


Figure 4.1: Architecture du pipeline RAG de PropIQ

4.2.2 Modèles d'intelligence artificielle mobilisés

PropIQ utilise plusieurs modèles d'intelligence artificielle afin de répondre aux différentes fonctionnalités de l'application. Le choix du modèle dépend de la nature de la tâche : traduction, transcription vocale, raisonnement juridique ou analyse de documents visuels.

Les modèles de raisonnement juridique sont accessibles à travers un routeur multi-LLM basé sur **OpenRouter**. Cette organisation permet de comparer plusieurs modèles et de laisser la possibilité de sélectionner celui qui correspond le mieux au besoin de l'utilisateur [14].

4.2.2.1 Traduction

Helsinki-NLP/opus-mt-en-ROMANCE Le modèle Helsinki-NLP/opus-mt-en-ROMANCE, développé par le groupe Helsinki-NLP, est utilisé pour la traduction des requêtes et des réponses. Il repose sur l'architecture MarianMT, basée sur un Transformer de type encodeur-décodeur [31].

Le modèle a été entraîné à partir du corpus OPUS, qui regroupe des textes parallèles dans plusieurs langues. La variante **en-ROMANCE** est destinée à la traduction de l'anglais vers plusieurs langues romanes, notamment le français, le portugais, l'espagnol

et l’italien [31].

Dans PropIQ, ce modèle permet notamment de faciliter les échanges entre la langue utilisée par l’utilisateur et le portugais, qui est la langue principale des documents réglementaires exploités par le système.

- **Architecture** : Transformer encodeur-décodeur basé sur MarianMT ;
- **Corpus d’entraînement** : corpus OPUS de textes parallèles multilingues ;
- **Rôle dans PropIQ** : assurer la cohérence linguistique entre les requêtes, les documents juridiques portugais et les réponses générées ;
- **Avantage retenu** : modèle spécialisé et relativement léger, adapté à une tâche de traduction ciblée.

4.2.2.2 Transcription vocale

Whisper PropIQ intègre également une fonctionnalité d’interaction vocale. Pour transformer la question orale de l’utilisateur en texte, l’application utilise Whisper, un modèle de reconnaissance automatique de la parole (*Automatic Speech Recognition*, ASR) développé par OpenAI [32, 33].

Whisper repose sur une architecture Transformer encodeur-décodeur et a été entraîné sur un vaste corpus audio multilingue et multitâche. Le modèle a notamment été entraîné sur environ 680 000 heures de données audio et leurs transcriptions [33].

Dans PropIQ, la transcription obtenue est ensuite transmise au pipeline de traitement du chatbot juridique. L’utilisateur peut ainsi poser une question oralement sans avoir à la saisir manuellement.

- **Architecture** : Transformer encodeur-décodeur de type séquence-à-séquence ;
- **Corpus d’entraînement** : environ 680 000 heures d’audio multilingue et multitâche ;
- **Capacités** : transcription automatique, détection de la langue et traitement de contenus audio multilingues ;

- **Rôle dans PropIQ** : convertir les questions vocales en texte afin de les intégrer au pipeline RAG ;
- **Avantage retenu** : disponibilité de plusieurs tailles de modèles, permettant d'adapter le compromis entre précision et temps de traitement.

4.2.2.3 Raisonnement juridique

Le raisonnement juridique constitue la partie centrale du système d'intelligence artificielle de PropIQ. Plusieurs modèles de langage ont été étudiés afin d'identifier ceux qui offrent les meilleures performances pour traiter des questions réglementaires.

La comparaison prend notamment en compte la capacité de raisonnement, la taille de la fenêtre de contexte, le type d'architecture ainsi que le compromis entre temps de réponse et qualité des résultats. Les modèles étudiés comprennent à la fois des architectures denses et des architectures de type *Mixture-of-Experts* (MoE).

Qwen3-235B-A22B Qwen3-235B-A22B, développé par Alibaba Cloud, est un modèle reposant sur une architecture *Mixture-of-Experts*. Il possède 235 milliards de paramètres au total, dont environ 22 milliards sont activés lors du traitement d'une requête [34].

Cette architecture permet de mobiliser différents experts en fonction de la question traitée tout en conservant une capacité importante de raisonnement.

- **Architecture** : Mixture-of-Experts (MoE), 235 milliards de paramètres au total et 22 milliards actifs ;
- **Fenêtre de contexte** : 128K tokens [34] ;
- **Particularité** : disponibilité d'un mode de raisonnement explicite (*thinking mode*) et d'un mode de réponse directe ;
- **Intérêt pour PropIQ** : la grande fenêtre de contexte permet de traiter un nombre important de passages juridiques récupérés par le RAG, notamment lorsque plusieurs textes réglementaires doivent être croisés.

Mistral Large 3 (2512) Mistral Large 3, développé par Mistral AI, repose sur une architecture *Mixture-of-Experts* multimodale. Le modèle comporte 675 milliards de paramètres au total, dont environ 41 milliards sont actifs [35].

Sa capacité multimodale permet également de traiter des informations visuelles, ce qui représente un intérêt pour les fonctionnalités de PropIQ liées à l’analyse de documents.

- **Architecture** : Mixture-of-Experts multimodale, 675 milliards de paramètres au total et 41 milliards actifs ;
- **Fenêtre de contexte** : 256K tokens [35] ;
- **Particularité** : capacité multimodale pour l’analyse d’images et de documents complexes ;
- **Intérêt pour PropIQ** : possibilité de combiner le raisonnement juridique et l’analyse documentaire au sein d’un même modèle.

GPT-4o GPT-4o, développé par OpenAI, est un modèle multimodal propriétaire capable de traiter différentes modalités, notamment le texte, l’image et l’audio. Son architecture interne n’est pas publiée [36].

Dans PropIQ, il constitue l’un des modèles pouvant être sélectionnés pour le traitement des questions juridiques. Ses performances générales en compréhension du langage et son temps de réponse en font une solution intéressante pour les scénarios nécessitant une interaction rapide.

- **Architecture** : modèle multimodal propriétaire ; architecture interne non divulguée ;
- **Fenêtre de contexte** : 128K tokens ;
- **Particularité** : modèle multimodal conçu pour traiter plusieurs types d’entrées ;
- **Intérêt pour PropIQ** : bonne capacité générale de compréhension et de génération, avec une latence adaptée à l’interaction avec l’utilisateur.

Llama 3.3 70B Instruct Llama 3.3 70B Instruct, développé par Meta, est un modèle Transformer dense de 70 milliards de paramètres. Il est principalement destiné au traitement du texte et au suivi des instructions [37].

Dans PropIQ, ce modèle présente un compromis intéressant entre les performances de raisonnement et le coût d'inférence, ce qui le rend adapté à une utilisation interactive du chatbot juridique.

- **Architecture** : Transformer dense de 70 milliards de paramètres ;
- **Fenêtre de contexte** : 128K tokens ;
- **Particularité** : modèle principalement orienté vers le traitement textuel et le suivi des instructions ;
- **Intérêt pour PropIQ** : compromis entre qualité des réponses, temps de traitement et coût d'inférence.

Phi-4 Phi-4, développé par Microsoft, est un modèle Transformer dense de 14 milliards de paramètres. Il a été conçu pour proposer de bonnes capacités de raisonnement malgré une taille plus réduite que les grands modèles de langage.

Dans PropIQ, il peut être utilisé pour des tâches de raisonnement moins complexes ou comme solution de repli lorsque la rapidité constitue un critère prioritaire.

- **Architecture** : Transformer dense de 14 milliards de paramètres ;
- **Fenêtre de contexte** : 16K tokens ;
- **Particularité** : modèle compact avec un accent mis sur les capacités de raisonnement ;
- **Intérêt pour PropIQ** : solution légère pour certaines requêtes, mais sa fenêtre de contexte limite son utilisation lorsque le RAG doit fournir un grand nombre de passages juridiques.

4.2.2.4 Analyse multimodale

PropIQ permet également à l'utilisateur de transmettre des documents sous forme d'images ou de fichiers numérisés. Des modèles vision-langage sont donc utilisés pour analyser les informations contenues dans ces documents.

Llama 3.2 11B Vision Instruct Llama 3.2 11B Vision Instruct est une variante multimodale de la famille Llama développée par Meta. Elle associe un modèle de langage à un composant dédié au traitement des informations visuelles [38].

Dans PropIQ, ce modèle est utilisé pour analyser les documents transmis par l'utilisateur, notamment les images et les documents PDF numérisés, afin d'en extraire les informations utiles au traitement de la demande.

- **Architecture** : modèle dense multimodal vision-langage de 11 milliards de paramètres ;
- **Fenêtre de contexte** : 128K tokens ;
- **Rôle dans PropIQ** : analyse des images et des documents numérisés transmis par l'utilisateur et extraction des informations pertinentes.

Qwen3-VL-32B-Instruct Qwen3-VL-32B-Instruct est un modèle vision-langage développé par Alibaba Cloud. Il associe des composants spécialisés dans le traitement des informations visuelles et textuelles afin de permettre une compréhension multimodale des documents.

Ce modèle est particulièrement adapté aux documents présentant une structure complexe, tels que les tableaux, les plans ou les documents administratifs comportant plusieurs éléments visuels.

- **Architecture** : modèle vision-langage multimodal ;
- **Fenêtre de contexte** : selon la configuration du modèle utilisée dans PropIQ ;
- **Rôle dans PropIQ** : analyse de documents visuels complexes, notamment les documents administratifs ou cadastraux comportant plusieurs pages et différents éléments de mise en page.

L'ensemble de ces modèles permet à PropIQ de couvrir plusieurs types d'interactions avec l'utilisateur. Les modèles de langage sont principalement dédiés au raisonnement juridique, tandis que les modèles spécialisés prennent en charge la traduction, la transcription et l'analyse des documents visuels. Le routeur multi-LLM apporte également une certaine flexibilité en permettant de choisir un modèle selon les besoins en termes de rapidité, de coût et de capacité de raisonnement.

4.2.3 Modélisation de l'indice de qualité de vie

En complément du chatbot juridique, PropIQ propose un indice de qualité de vie destiné à évaluer l'environnement d'un bien immobilier à partir de sa localisation. Cet indice combine des données géographiques issues d'OpenStreetMap/Overpass (établissements scolaires, établissements de santé, commerces, espaces verts, transports en commun) et une donnée de qualité de l'air issue de l'API Open-Meteo.

4.2.3.1 Critères et pondération

Six critères sont pris en compte, chacun associé à un poids fixe exprimé en pourcentage, la somme des poids étant égale à 100 :

Table 4.1: Pondération des critères de l'indice de qualité de vie

Critère	Poids
Établissements scolaires	20%
Établissements de santé	20%
Qualité de l'air	20%
Commerces	15%
Espaces verts	15%
Transports en commun	10%

Cette pondération traduit un choix méthodologique consistant à accorder la priorité la plus élevée aux critères jugés essentiels pour la qualité de vie et la santé des habitants (écoles, santé, air), un poids intermédiaire aux critères de confort (commerces, espaces verts), et un poids plus faible aux transports, considérés comme un facteur secondaire

par rapport aux précédents. Ces poids ont été fixés empiriquement lors de la conception du module ; ils ne résultent pas d’une enquête utilisateur ou d’une méthode de pondération formalisée (par exemple une analyse multicritère de type AHP), ce qui constitue une limite reconnue de l’indice actuel et une piste d’amélioration pour une validation ultérieure.

4.2.3.2 Calcul des scores par critère

Pour les cinq critères géographiques (écoles, santé, commerces, espaces verts, transports), le nombre de points d’intérêt recensés dans un rayon de 3 km autour du bien est d’abord obtenu via l’API Overpass. Ce nombre est ensuite converti en un score compris entre 0 et 100 à l’aide d’une fonction de normalisation par paliers, propre à chaque critère. Par exemple, pour le critère « écoles », l’absence d’établissement dans le rayon considéré donne un score de 0, la présence d’un établissement donne 40, de deux établissements 65, de trois 85, et de cinq établissements ou plus 100. Chaque critère dispose ainsi de sa propre grille de paliers, définie en fonction du nombre de points d’intérêt jugé raisonnable dans un contexte résidentiel.

Le critère de qualité de l’air est traité différemment : la valeur de concentration en particules fines (PM2.5, ou PM10 en absence de PM2.5) obtenue via Open-Meteo est convertie en score selon une relation inverse, une concentration plus faible donnant un score plus élevé. Les paliers utilisés sont les suivants : une concentration inférieure ou égale à 5 $\mu\text{g}/\text{m}^3$ donne un score de 100, inférieure ou égale à 15 $\mu\text{g}/\text{m}^3$ donne 85, inférieure ou égale à 25 $\mu\text{g}/\text{m}^3$ donne 65, inférieure ou égale à 35 $\mu\text{g}/\text{m}^3$ donne 40, inférieure ou égale à 50 $\mu\text{g}/\text{m}^3$ donne 20, et au-delà, un score minimal de 5 est appliqué.

4.2.3.3 Gestion des données manquantes

Lorsque la donnée de qualité de l’air n’est pas disponible (échec de l’appel à l’API Open-Meteo), un score neutre de 50 est attribué à ce critère plutôt qu’un score nul, afin d’éviter de pénaliser artificiellement un bien pour lequel l’information n’a simplement pas pu être récupérée. Ce choix privilégie la neutralité par rapport à une valeur par défaut pessimiste ou optimiste. Pour les critères géographiques, l’absence de point d’intérêt dans le rayon de recherche est en revanche traitée comme une donnée valide

(score de 0), dans la mesure où l’absence de résultat Overpass reflète généralement une absence réelle d’infrastructure à proximité plutôt qu’une erreur technique.

4.2.3.4 Score global

Le score global de l’indice de qualité de vie, noté S , est calculé comme la somme pondérée des scores obtenus pour chacun des six critères :

$$S = \sum_{i=1}^6 \frac{s_i \times p_i}{100} \quad (4.1)$$

où s_i est le score du critère i (compris entre 0 et 100) et p_i son poids, exprimé en pourcentage (Tableau 4.1). Le score global S est ainsi lui-même compris entre 0 et 100.

Ce score est ensuite traduit en une recommandation qualitative selon les seuils suivants : un score supérieur ou égal à 75 correspond à une recommandation d’achat (« excellente qualité de vie »), un score compris entre 55 et 74 correspond à une recommandation neutre, un score compris entre 35 et 54 appelle à la prudence et invite à approfondir l’analyse, et un score inférieur à 35 déconseille l’achat. Ces seuils ont, comme la pondération des critères, été définis empiriquement et n’ont pas fait l’objet d’une validation statistique sur un échantillon de biens ; leur affinement à partir de retours utilisateurs ou d’une comparaison avec des indicateurs immobiliers existants (par exemple les écarts de prix au m² selon la zone) constitue une perspective d’amélioration pour ce module.

4.3 Évaluation des modèles de langage

4.3.1 Protocole et environnement d’évaluation

Une évaluation comparative a été menée sur cinq modèles de raisonnement juridique afin d’identifier celui qui répond le mieux aux contraintes du chatbot PropIQ. Les expérimentations ont été suivies avec MLflow, ce qui permet d’enregistrer les paramètres, les métriques et les résultats associés à chaque expérimentation [30].

Le corpus de test est constitué de quatre questions juridiques représentatives, chacune

correspondant à l'un des quatre types de biens pris en charge par PropIQ (résidentiel, commercial, terrain, hébergement touristique). Chaque question a été soumise à chacun des cinq modèles évalués, ce qui porte le nombre total de réponses analysées à vingt ($4 \text{ questions} \times 5 \text{ modèles}$). Ce nombre limité de scénarios par modèle constitue une contrainte assumée de cette phase d'évaluation : les cinq modèles ont été interrogés via l'API OpenRouter dans le cadre d'un plan d'utilisation gratuit, dont les limites de quota et de fréquence de requêtes ne permettaient pas de multiplier davantage les appels sans dégrader la fiabilité des mesures de temps de réponse. Le choix d'une question par type de bien a néanmoins permis de couvrir les principaux cas d'usage de l'application tout en respectant cette contrainte. L'extension de ce protocole à un nombre plus important de scénarios, une fois l'application déployée sur un plan API payant, constitue une piste d'amélioration identifiée pour fiabiliser davantage les conclusions, notamment concernant le taux d'hallucination.

Pour chaque modèle, trois dimensions principales ont été considérées : le temps de réponse moyen, la pertinence des réponses générées (*answer_relevance*) et le nombre d'hallucinations détectées. Les hallucinations correspondent ici aux réponses contenant notamment un article juridique incorrect, une référence réglementaire fabriquée, ou une affirmation non appuyée par le contexte juridique récupéré. Leur détection repose sur une approche *LLM-as-judge* : un modèle de langage évalue automatiquement la fidélité (*faithfulness*) de chaque réponse par rapport au contexte récupéré et signale une hallucination lorsqu'une affirmation n'est pas justifiée par ce contexte.

4.3.2 Métriques retenues

Les critères utilisés pour comparer les modèles sont les suivants :

- **Temps de réponse moyen (s)** : temps moyen entre l'envoi de la question et la réception de la réponse complète. Il prend en compte la recherche vectorielle et la génération de la réponse ;
- **Pertinence des réponses (*answer_relevance*)** : score compris entre 0 et 1, calculé selon une approche *LLM-as-judge*. Le modèle juge reçoit la question posée et la réponse générée, puis attribue un score selon une grille fixe (1,0 : réponse complète et directe ; 0,7 : réponse globalement adaptée mais incomplète

sur certains aspects ; 0,4 : réponse partielle ; 0,1 : réponse faiblement liée à la question ; 0,0 : réponse hors sujet). L'appel au modèle juge est effectué à température nulle (`temperature = 0.0`) afin de limiter la variabilité du score d'une exécution à l'autre, la réponse étant structurée en JSON (score et justification courte) pour en faciliter l'exploitation automatique ;

- **Hallucinations détectées** : nombre de réponses, parmi les quatre réponses analysées pour chaque modèle, pour lesquelles le juge LLM signale une affirmation non fondée sur le contexte juridique récupéré (score de fidélité faible et indicateur d'hallucination positif).

4.3.3 Résultats comparatifs

Le tableau 4.2 présente les résultats obtenus pour les cinq modèles évalués.

Table 4.2: Comparaison des modèles de langage selon le temps de réponse, la pertinence et les hallucinations détectées

Modèle	Temps moyen (s)	Pertinence	Hallucinations (/4)
GPT-4o	8,72	0,78	0
Llama 3.3 70B	9,57	0,89	0
Mistral Large 3 (2512)	14,64	0,91	1
Phi-4	11,72	0,86	3
Qwen3-235B-A22B	18,12	0,93	2

4.3.4 Analyse et choix du modèle

Les résultats montrent des différences importantes entre les modèles, notamment en termes de rapidité, de pertinence et de fiabilité.

GPT-4o obtient le temps de réponse moyen le plus faible, avec 8,72 secondes, et aucune hallucination détectée dans l'échantillon analysé. Cependant, son score de pertinence de 0,78 est le plus faible parmi les cinq modèles évalués. Ce résultat indique que,

dans certains scénarios complexes, les réponses peuvent être moins complètes ou moins adaptées à la question juridique posée.

À l’opposé, Qwen3-235B-A22B obtient le meilleur score de pertinence avec 0,93. Il présente toutefois le temps de réponse le plus élevé, avec 18,12 secondes, ainsi que deux hallucinations sur les quatre réponses analysées.

Llama 3.3 70B présente un compromis plus équilibré. Son temps de réponse moyen est de 9,57 secondes, son score de pertinence atteint 0,89 et aucune hallucination n’a été détectée dans l’échantillon étudié. Il offre ainsi une combinaison intéressante entre rapidité et qualité des réponses.

Mistral Large 3 (2512) obtient également un score de pertinence élevé de 0,91, mais avec un temps de réponse supérieur à celui de Llama 3.3 70B et une hallucination détectée. Phi-4 présente quant à lui un temps de réponse intermédiaire, mais le nombre d’hallucinations détectées est plus important, avec trois réponses concernées sur quatre.

Au regard de ces résultats, **Llama 3.3 70B Instruct** a été retenu comme modèle par défaut pour la production. Ce choix repose principalement sur son équilibre entre les trois critères étudiés. Il fournit une meilleure pertinence que GPT-4o, avec un temps de réponse seulement légèrement supérieur, tout en conservant l’absence d’hallucination observée sur l’échantillon évalué. Comparé à Qwen3-235B-A22B, il offre un temps de réponse nettement plus faible tout en conservant un niveau de pertinence élevé.

Les autres modèles restent disponibles à travers le routeur multi-LLM. Cette possibilité permet notamment de privilégier Qwen3-235B-A22B lorsque la priorité est donnée à la profondeur du raisonnement, ou GPT-4o lorsque la rapidité de réponse est le principal critère recherché.

4.4 Conclusion

Ce chapitre a présenté les principales composantes de modélisation et d’évaluation de PropIQ. Le pipeline RAG constitue le cœur du chatbot juridique en combinant la recherche sémantique dans le corpus réglementaire avec la génération de réponses par plusieurs modèles de langage.

Les différents modèles utilisés dans la solution ont également été présentés selon leur

rôle : traduction, transcription vocale, raisonnement juridique et analyse multimodale. Cette organisation permet à PropIQ de prendre en charge différents modes d’interaction tout en adaptant le modèle utilisé à la nature de la tâche.

Enfin, l’évaluation comparative de cinq modèles de raisonnement juridique a permis de mettre en évidence un compromis entre rapidité, pertinence et fiabilité. Sur la base des résultats obtenus, Llama 3.3 70B Instruct a été retenu comme modèle par défaut, tandis que les autres modèles restent accessibles à travers le routeur multi-LLM.

Le chapitre suivant est consacré aux tests fonctionnels et à la validation de la solution PropIQ. Il permettra de vérifier le comportement de l’application dans différents scénarios et d’évaluer son adéquation avec les besoins définis précédemment.

CHAPITRE 5

Déploiement

5.1 Introduction

Ce cinquième chapitre correspond à la phase *Deployment* de la méthodologie CRISP-DM. Il présente la mise en œuvre technique de PropIQ et les choix réalisés pour transformer les différents composants développés précédemment en une solution fonctionnelle.

Dans un premier temps, les principales technologies utilisées sont présentées, en mettant l'accent sur leur rôle dans l'architecture globale. Une comparaison entre FAISS et Pinecone permet ensuite de justifier le choix de la base vectorielle utilisée pour le pipeline RAG.

Le chapitre décrit également l'architecture de la solution à travers les diagrammes UML, l'architecture globale, le pipeline NLP/RAG et la chaîne MLOps. La réalisation du backend FastAPI, la gestion des utilisateurs, des quotas, des paiements et des interactions multimodales sont ensuite détaillées.

Enfin, les interfaces de l'application mobile Flutter sont présentées afin d'illustrer le résultat final obtenu.

5.2 Choix technologiques

Le cahier des charges initial prévoyait une application web basée sur React.js, associée à PostgreSQL et à une solution de recherche vectorielle telle que FAISS ou ChromaDB. Au cours du développement, ces choix ont été adaptés afin de mieux répondre aux besoins identifiés pour PropIQ, notamment l'utilisation sur appareil mobile et les contraintes liées à la recherche sémantique.

L'architecture finale repose ainsi sur Flutter pour l'application mobile, FastAPI pour le backend, SQLite pour le stockage relationnel et Pinecone pour la recherche vectorielle. Plusieurs services complémentaires sont utilisés pour l'authentification, le paiement, l'accès aux modèles de langage, la conteneurisation, l'intégration continue et la supervision.

5.2.1 Flutter



Figure 5.1: Logo Flutter

Flutter est un framework open source développé par Google permettant de construire des applications multiplateformes à partir d'une base de code commune. La documentation officielle fournit notamment les composants, les API et les recommandations nécessaires au développement d'applications Flutter [39].

Dans le cadre de PropIQ, Flutter a été retenu afin de proposer une application mobile unique. Ce choix répond davantage au contexte d'utilisation prévu pour les clients et les agents immobiliers de Remax Wise, qui peuvent accéder aux fonctionnalités de l'application directement depuis leur smartphone.

5.2.2 FastAPI



Figure 5.2: Logo FastAPI

FastAPI est un framework Python destiné au développement d'API web. Il permet notamment de définir des routes, de valider les données d'entrée et de générer automatiquement une documentation interactive basée sur OpenAPI [40].

Dans PropIQ, FastAPI constitue la couche backend qui assure la communication entre l'application mobile et les différents services de la solution. Il expose notamment les fonctionnalités d'authentification, de chatbot juridique, d'analyse documentaire, de traitement vocal, de qualité de vie, de gestion des crédits et de paiement.

5.2.3 Firebase Authentication



Figure 5.3: Logo Firebase

Firebase Authentication est utilisé pour gérer l'inscription et l'authentification des utilisateurs. Le service prend notamment en charge l'authentification par adresse e-mail et mot de passe ainsi que les fournisseurs d'identité tels que Google Sign-In [15].

Dans PropIQ, le service assure la gestion de l'identité des utilisateurs et fournit les jetons d'authentification nécessaires. Le backend FastAPI vérifie ensuite les jetons

transmis par l'application avant d'autoriser l'accès aux fonctionnalités protégées.

5.2.4 FAISS

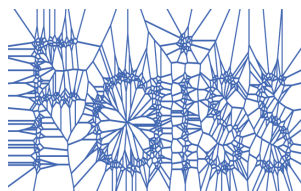


Figure 5.4: Logo FAISS

FAISS est une bibliothèque développée par Meta AI pour effectuer efficacement des recherches de similarité et du clustering sur des vecteurs denses. Elle prend notamment en charge différentes mesures de distance et peut être utilisée avec des accélérations matérielles [41].

Elle a été utilisée au début du projet pour indexer localement les embeddings issus du corpus juridique.

Ce choix était adapté aux premières expérimentations grâce à sa simplicité d'intégration et à son fonctionnement local. Cependant, son utilisation impliquait que la persistance, la gestion et la mise à jour des index soient prises en charge au niveau de l'application. Ces contraintes ont conduit à étudier une solution vectorielle managée et à migrer progressivement vers Pinecone.

5.2.5 Pinecone



Figure 5.5: Logo Pinecone

Pinecone est une base de données vectorielle destinée aux applications d'intelligence artificielle et notamment à la recherche sémantique et à la récupération de connaissances [26].

Dans PropIQ, Pinecone remplace FAISS pour stocker les embeddings des documents juridiques et effectuer la recherche sémantique du pipeline RAG. Son approche managée fournit des mécanismes de gestion des index, de recherche sémantique, de mise à jour des données et de suivi des performances.

Cette solution permet ainsi de réduire la charge liée à la maintenance locale de l'index et de disposer d'une infrastructure plus adaptée à une évolution du volume de données.

5.2.6 OpenRouter



Figure 5.6: Logo OpenRouter

OpenRouter fournit une API unifiée permettant d'accéder à de nombreux modèles d'intelligence artificielle à travers une interface commune [14]. Son API permet également de récupérer la liste des modèles disponibles et leurs caractéristiques.

Dans PropIQ, cette couche permet notamment d'utiliser et de comparer des modèles issus des familles Mistral, Qwen, GPT, Llama et Phi.

Cette approche évite de dépendre directement d'une intégration spécifique à chaque fournisseur et facilite le changement de modèle selon les besoins de l'application. Elle permet également de proposer à l'utilisateur plusieurs modèles pour le traitement des questions juridiques.

5.2.7 SQLite



Figure 5.7: Logo SQLite

SQLite est un moteur de base de données relationnelle embarqué, auto-contenu et sans serveur séparé [42].

Dans PropIQ, SQLite est utilisé pour stocker les informations applicatives telles que les données des utilisateurs, les abonnements, l'historique des interactions et certains paramètres. Ce choix est adapté au périmètre actuel du projet et permet de conserver une architecture simple.

Une migration vers une base de données client-serveur telle que PostgreSQL reste possible si les besoins en charge, en concurrence ou en déploiement distribué augmentent.

5.2.8 Stripe



Figure 5.8: Logo Stripe

Stripe est utilisé pour la gestion des paiements liés aux abonnements Premium. Stripe Checkout fournit notamment une interface de paiement préconstruite et prend en charge les paiements ponctuels ainsi que les abonnements [43].

Le système s'appuie également sur des webhooks afin de synchroniser l'état du paiement avec le backend. Cette approche permet de confirmer le résultat d'une transaction directement côté serveur plutôt que de dépendre uniquement de la redirection de l'utilisateur après paiement [44].

Le projet prévoit également l'utilisation de Konnect pour le marché local. Cette combinaison permet d'adapter le système de paiement aux différents contextes d'utilisation de PropIQ.

5.2.9 Docker



Figure 5.9: Logo Docker

Docker est utilisé pour conteneuriser le backend et ses dépendances. Les conteneurs permettent d'exécuter les processus dans des environnements isolés, avec leur propre système de fichiers et leur propre configuration réseau [45].

Cette conteneurisation facilite le passage entre les différents environnements de développement, de test et de déploiement et réduit les différences liées à la configuration de la machine hôte.

5.2.10 GitHub Actions

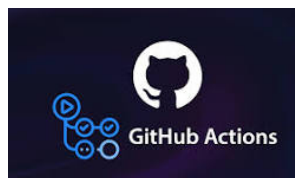


Figure 5.10: Logo GitHub Actions

GitHub Actions permet d'automatiser les workflows de développement et d'intégration continue directement depuis le dépôt GitHub [46].

Dans PropIQ, les workflows peuvent être déclenchés lors d'un `push` ou d'une `pull request` et exécutent différentes vérifications avant la publication de l'image Docker.

Les workflows comprennent notamment les étapes de vérification du code, de tests et de construction de l'application. Les informations sensibles nécessaires au processus sont stockées dans les secrets GitHub, qui permettent de conserver les valeurs confidentielles séparément du code source [47].

5.2.11 MLflow



Figure 5.11: Logo MLflow

MLflow est utilisé pour assurer le suivi des expérimentations liées aux modèles et au pipeline RAG. MLflow Tracking permet notamment d'enregistrer les paramètres, les métriques, les versions de code et les artefacts associés aux différentes exécutions [30]. Dans PropIQ, MLflow facilite notamment la comparaison des modèles de langage et le suivi des performances du système au cours du développement.

5.2.12 Elasticsearch et Kibana



Figure 5.12: Logos Elasticsearch et Kibana

Elasticsearch est utilisé comme moteur d'indexation et de recherche pour centraliser les informations de supervision. Kibana fournit l'interface permettant d'explorer les

données et de construire des tableaux de bord.

Dans PropIQ, cette combinaison permet notamment de suivre les logs et certaines métriques applicatives afin de faciliter la surveillance du système et l'identification des anomalies. Kibana permet notamment de rechercher et filtrer les logs ingérés dans Elasticsearch [48].

5.2.13 Ngrok



Figure 5.13: Logo Ngrok

Ngrok est utilisé comme outil de tunneling permettant de rendre temporairement accessible sur Internet un service exécuté localement. La documentation officielle décrit notamment le transfert du trafic public vers un service local à travers un endpoint Ngrok [49].

Dans PropIQ, il a principalement été utilisé pendant les phases de développement et de test. Il permettait notamment de rendre l'API FastAPI locale accessible depuis l'application Flutter installée sur un appareil physique ou depuis certains services externes nécessitant une URL accessible publiquement.

- **Rôle dans PropIQ** : exposition temporaire de l'API backend locale pour les tests et les démonstrations ;
- **Fonctionnement** : création d'un tunnel permettant de transférer les requêtes reçues sur un endpoint public vers le service local ;
- **Avantage** : possibilité de tester les interactions entre l'application mobile, le backend et certains services externes sans déployer immédiatement l'API sur un serveur distant.

5.3 Comparaison entre FAISS et Pinecone

Le choix de Pinecone résulte de l'évolution du projet depuis les premières expérimentations locales vers une architecture plus adaptée au déploiement. FAISS a été utilisé initialement pour effectuer les recherches vectorielles en local, puis ses contraintes de gestion locale des index ont conduit à étudier une solution managée.

Le tableau 5.1 présente les principales différences considérées lors de cette évolution.

Table 5.1: Comparaison entre FAISS et Pinecone

Critère	FAISS	Pinecone
Type	Bibliothèque de recherche vectorielle locale	Base vectorielle managée
Hébergement	Géré par l'application	Service géré par le fournisseur
Persistance	Gestion locale des index	Gestion des index par le service
Scalabilité	Dépend de l'infrastructure utilisée	Fonctionnalités de passage à l'échelle proposées par le service
Mise à jour des données	Gestion par l'application	API de gestion et de mise à jour des données
Maintenance	À la charge du développeur	Réduite grâce au service managé
Recherche sémantique	Supportée	Supportée avec des fonctionnalités dédiées

FAISS est particulièrement adapté aux expérimentations locales et fournit des algorithmes efficaces de recherche de similarité sur des vecteurs denses [41]. Pinecone, de son côté, fournit une infrastructure spécialisée pour les applications de recherche sémantique et de récupération de connaissances, avec des fonctionnalités de gestion des index et d'optimisation des performances [26].

Dans le cadre de PropIQ, les expérimentations et les besoins de déploiement ont conduit

à retenir Pinecone pour la version finale du pipeline RAG. Ce choix permet notamment de réduire la charge de maintenance des index et de disposer d’une infrastructure plus adaptée à l’évolution du corpus juridique.

5.4 Diagrammes de conception et d’architecture

5.4.1 Diagrammes UML

5.4.1.1 Diagramme de cas d’utilisation

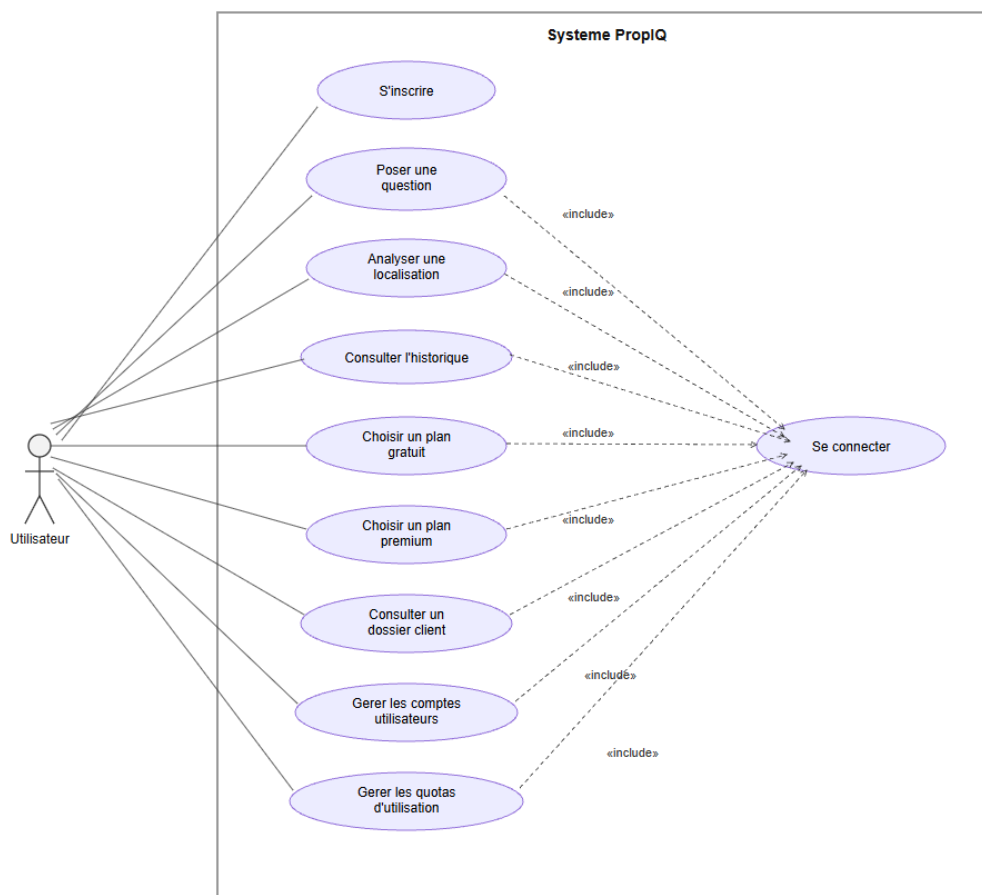


Figure 5.14: Diagramme de cas d’utilisation de PropIQ

La figure 5.14 présente les principales interactions entre l’acteur Utilisateur et le système PropIQ.

L’acteur *Utilisateur* peut s’inscrire auprès du système (*S’inscrire*) et s’y connecter (*Se connecter*). Il peut ensuite accéder aux principales fonctionnalités de l’application :

poser une question au chatbot juridique, analyser une localisation, consulter l’historique de ses analyses, choisir un plan gratuit ou premium, consulter un dossier client, gérer les comptes utilisateurs, ou encore gérer les quotas d’utilisation.

À l’exception de l’inscription, chacun de ces cas d’utilisation intègre une relation d’inclusion («*include*») vers le cas d’utilisation *Se connecter*, traduisant la nécessité d’une authentification préalable pour y accéder.

5.4.1.2 Diagramme de classes

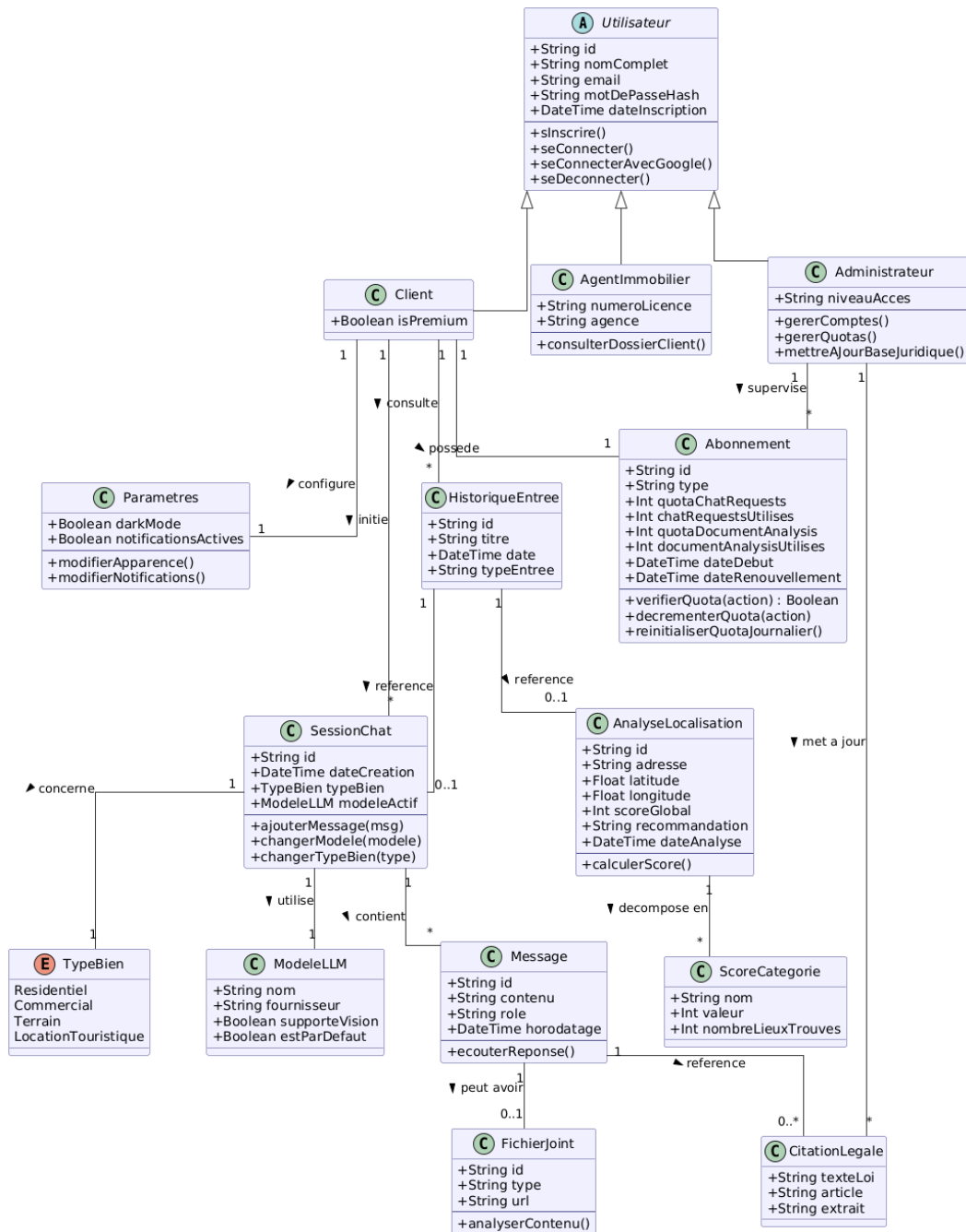


Figure 5.15: Diagramme de classes de PropIQ

Le diagramme de classes présenté dans la figure 5.15 décrit la structure objet de PropIQ. La classe abstraite *Utilisateur* regroupe les attributs et méthodes communs aux différents profils. Elle est spécialisée par les classes *Client*, *AgentImmobilier* et *Administrateur*.

Le client est associé à un abonnement, à ses paramètres et à son historique. L'historique

peut contenir des sessions de discussion ou des analyses de localisation. Une session de discussion contient des messages auxquels peuvent être associés des fichiers et des citations légales.

L’analyse de localisation est composée de plusieurs scores correspondant aux critères utilisés pour évaluer la qualité de vie. L’administrateur assure quant à lui la supervision des utilisateurs, des abonnements et des informations juridiques.

5.4.2 Architecture globale

La solution PropIQ repose sur une architecture organisée autour de trois niveaux principaux, comme l’illustre la figure 5.16.

Le premier niveau correspond à l’application mobile développée avec Flutter. Le deuxième niveau est constitué du backend FastAPI, qui assure la logique métier et l’exposition des services REST. Le troisième niveau regroupe les services de données et d’intelligence artificielle, notamment Pinecone, SQLite et les modèles de langage accessibles via OpenRouter.

Les composants nécessaires au fonctionnement du backend sont également regroupés dans des conteneurs Docker afin de faciliter leur exécution et leur déploiement.

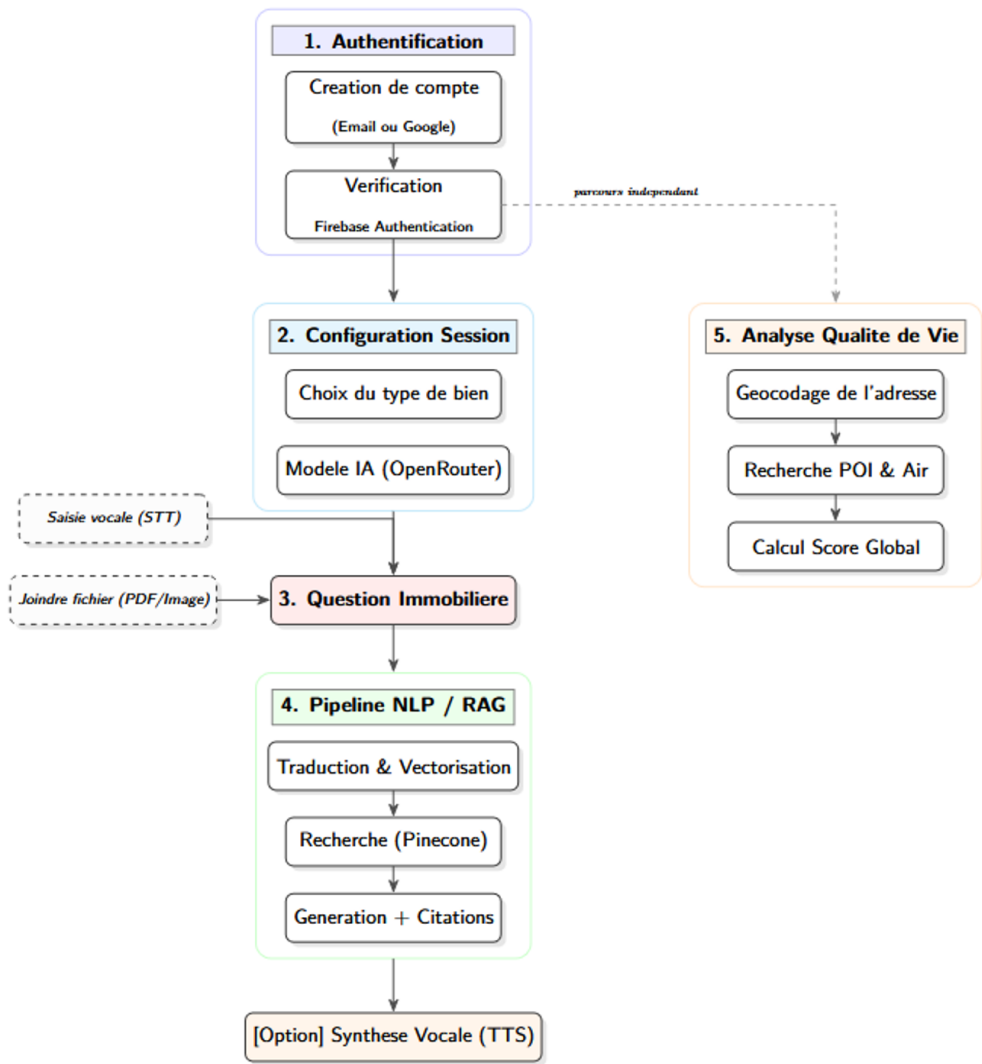


Figure 5.16: Architecture globale de la solution PropIQ

5.4.3 Architecture du pipeline NLP et RAG

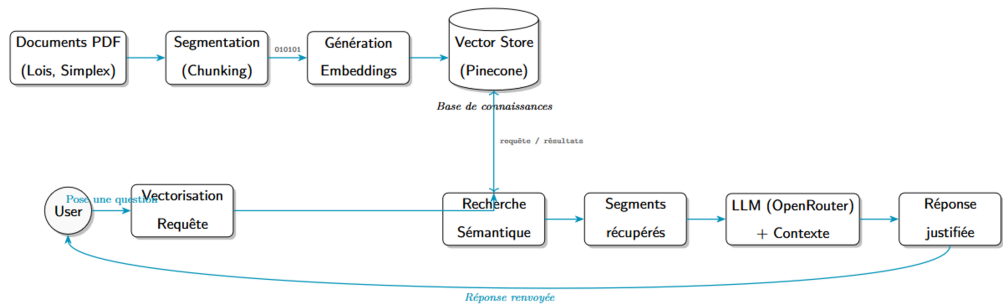


Figure 5.17: Architecture du pipeline NLP et RAG de PropIQ

Le pipeline NLP et RAG présenté dans la figure 5.17 regroupe les principales étapes permettant de transformer une question utilisateur en réponse juridique.

1. **Ingestion** : collecte des documents juridiques officiels portant notamment sur le cadre Simplex et les réglementations applicables au secteur immobilier ;
2. **Prétraitement** : nettoyage, normalisation et suppression des doublons, suivis de la segmentation des documents en unités adaptées à la recherche sémantique ;
3. **Vectorisation** : transformation des segments juridiques en représentations vectorielles et stockage des embeddings dans Pinecone ;
4. **Recherche sémantique** : transformation de la requête utilisateur en représentation vectorielle, puis récupération des passages les plus proches dans l'index ;
5. **Génération augmentée** : transmission des passages récupérés au modèle de langage sélectionné via OpenRouter afin de générer une réponse fondée sur le contexte juridique disponible ;
6. **Décision automatisée** : classification de la situation étudiée selon trois catégories : achat légalement possible, possible sous conditions ou non conforme.

Cette chaîne permet de relier directement la question de l'utilisateur aux documents réglementaires utilisés pour construire la réponse.

5.5 Pipeline MLOps

La mise en œuvre de la couche MLOps a été organisée en sept phases successives. Cette démarche permet de passer d'un code de développement à un système mieux structuré, automatisé et supervisé.

Les sept phases sont les suivantes :

1. modularisation du code ;
2. automatisation locale avec Makefile ;

3. exposition des services via FastAPI ;
4. suivi des expérimentations avec MLflow ;
5. conteneurisation avec Docker ;
6. intégration continue avec GitHub Actions ;
7. supervision applicative.

La figure 5.18 présente l'organisation générale de cette chaîne.

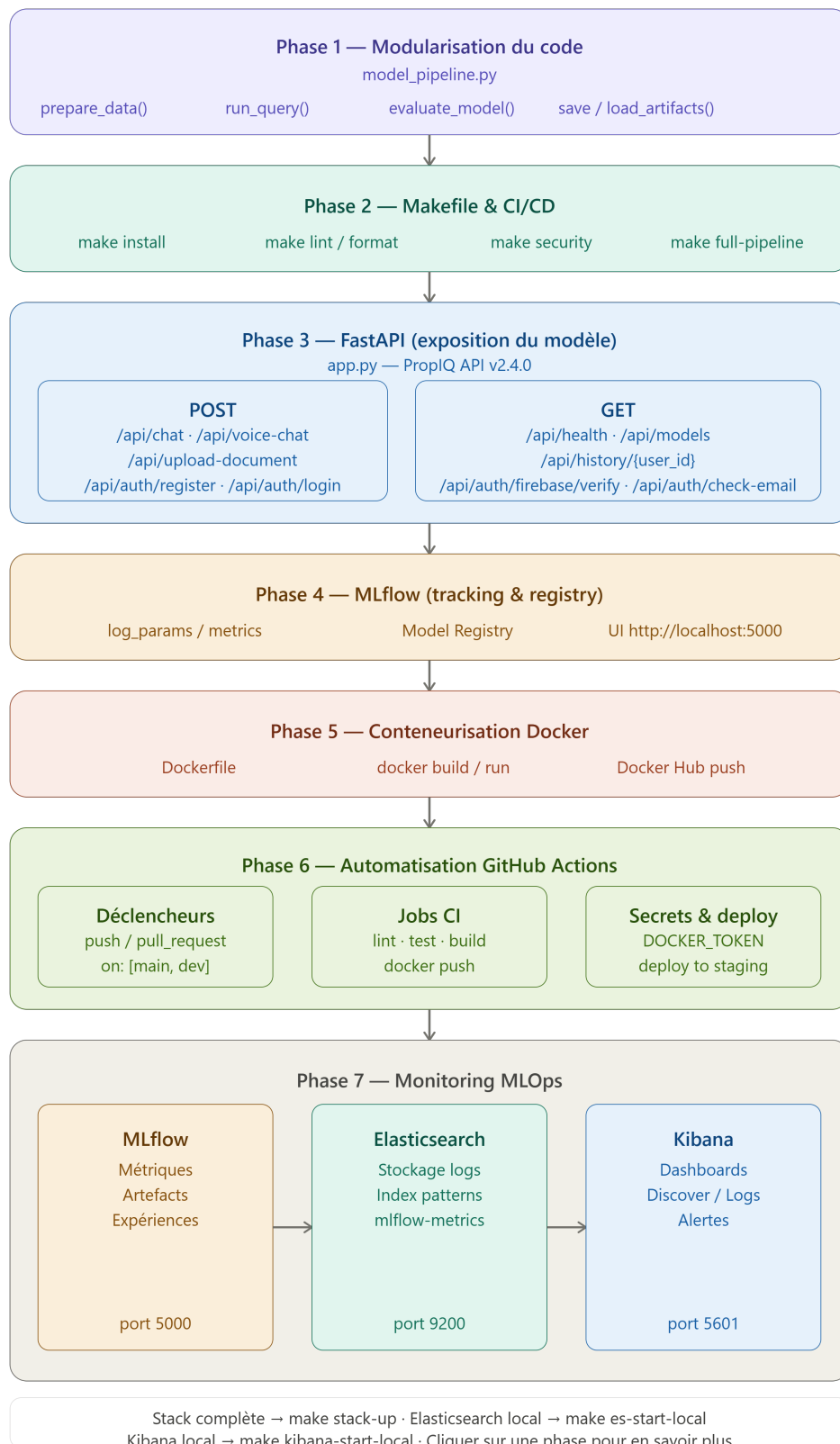


Figure 5.18: Architecture de la couche MLOps

5.5.0.1 Phase 1 – Modularisation du code

Le pipeline de modélisation a été réorganisé dans un module dédié nommé `model_pipeline.py`. Les différentes opérations sont séparées en fonctions afin de faciliter leur réutilisation et leur maintenance.

Les principales fonctions sont :

- `prepare_data()` : préparation des données ;
- `run_query()` : exécution des recherches sémantiques ;
- `evaluate_model()` : évaluation des performances ;
- `save_artifacts()` et `load_artifacts()` : sauvegarde et chargement des artefacts.

Cette organisation constitue la base des étapes d'automatisation présentées dans les phases suivantes.

5.5.0.2 Phase 2 – Makefile et automatisation locale

Un fichier `Makefile` regroupe les commandes utilisées régulièrement pendant le développement.

- `make install` : installation des dépendances ;
- `make lint` et `make format` : vérification et formatage du code ;
- `make security` : analyse des vulnérabilités ;
- `make full-pipeline` : exécution de l'ensemble des étapes de validation.

Cette automatisation permet de standardiser les opérations courantes et de réduire les manipulations manuelles avant l'intégration du code.

5.5.0.3 Phase 3 – Exposition des services via FastAPI

Les services du système sont exposés à travers l'API FastAPI, notamment dans le fichier `app.py`. La version indiquée dans la documentation est PropIQ API v2.4.0.

Les principales routes sont réparties comme suit :

- **Routes POST** : `/api/chat`, `/api/voice-chat`, `/api/upload-document`, `/api/auth/register` et `/api/auth/login` ;
- **Routes GET** : `/api/health`, `/api/models`, `/api/history/{user_id}`, `/api/auth/firebase` et `/api/auth/check-email`.

FastAPI constitue ainsi le point d'accès entre l'application Flutter et les différents services du backend.

5.5.0.4 Phase 4 – Suivi expérimental avec MLflow

MLflow est utilisé pour enregistrer les paramètres, les métriques et les artefacts associés aux expérimentations. Les fonctions `log_params` et `log_metrics` permettent de conserver les résultats obtenus lors des différents essais [30].

Le projet intègre également plusieurs composants dédiés :

- **RAGMetrics** : calcul des métriques liées à la qualité du pipeline RAG ;
- **DriftDetector** : détection d'une éventuelle évolution des performances dans le temps ;
- **MetricsAggregator** : regroupement et consolidation des indicateurs.

MLflow a d'abord été utilisé pour suivre cinq expérimentations de référence. Le dispositif a ensuite été étendu afin de permettre le suivi et la comparaison des sept modèles de langage accessibles via OpenRouter.

5.5.0.5 Phase 5 – Conteneurisation avec Docker

Le backend est empaqueté dans une image Docker définie à partir d'un `Dockerfile`. L'image peut être construite et exécutée localement à l'aide des commandes `docker build` et `docker run`, puis publiée sur Docker Hub avec `docker push`.

La conteneurisation permet de conserver un environnement logiciel cohérent lors des différentes étapes du déploiement. Docker fournit également des mécanismes d'isolation des processus, du système de fichiers et du réseau [45].

5.5.0.6 Phase 6 – Automatisation CI/CD avec GitHub Actions

L'intégration continue est assurée par GitHub Actions. Les workflows sont déclenchés lors des événements configurés sur les branches concernées, notamment `main` et `dev`.

La chaîne automatisée comprend plusieurs étapes :

- vérification du code avec le *lint* ;
- exécution des tests ;
- construction de l'image Docker ;
- publication de l'image ;
- déploiement vers l'environnement de *staging*.

Les informations sensibles utilisées par le workflow, comme le jeton `DOCKER_TOKEN`, sont stockées dans les secrets GitHub afin d'éviter leur présence directe dans le code source [47].

5.5.0.7 Phase 7 – Supervision applicative

La dernière phase concerne la supervision de l'application et du pipeline.

- **MLflow** centralise les métriques et les artefacts liés aux expérimentations ;
- le **middleware de monitoring FastAPI** collecte des informations telles que la latence, le taux d'erreur et le nombre de requêtes, puis les transmet à **Elasticsearch** ;
- **Elasticsearch** assure l'indexation et le stockage des informations de supervision ;
- **Kibana** permet d'explorer les logs, de construire des tableaux de bord et de configurer des alertes.

La stack peut être démarrée à l'aide de `make stack-up`, ou avec les commandes spécifiques `make es-start-local` et `make kibana-start-local`. Kibana permet notamment de rechercher et filtrer les logs collectés dans Elasticsearch [48].

5.6 Déploiement technique

Le déploiement technique de PropIQ repose sur l'intégration des différents composants présentés précédemment. Le backend FastAPI constitue le point central de communication entre l'application mobile et les services externes.

L'application mobile communique avec l'API à travers les routes REST. Le backend traite ensuite la requête en fonction de sa nature : recherche juridique, analyse d'un document, interaction vocale, analyse de qualité de vie, gestion du compte ou paiement.

Les composants nécessaires au fonctionnement du backend sont regroupés dans l'environnement Docker, tandis que les services externes tels que Firebase, Pinecone, OpenRouter et les plateformes de paiement sont appelés à travers leurs interfaces respectives.

5.7 Réalisation du backend FastAPI

5.7.1 Organisation générale

Le backend de PropIQ a été développé avec FastAPI. Il regroupe les services nécessaires au fonctionnement de l'application et expose les fonctionnalités sous forme de routes REST.

Les principales fonctionnalités couvertes sont l'authentification, le chatbot juridique, le traitement vocal, l'analyse documentaire, l'historique, l'évaluation des réponses, l'analyse de qualité de vie, la gestion des crédits ainsi que les abonnements et les paiements.

La figure 5.19 présente l'interface Swagger/OpenAPI générée automatiquement par FastAPI. Elle permet de visualiser les routes disponibles et de tester les différentes opérations de l'API.

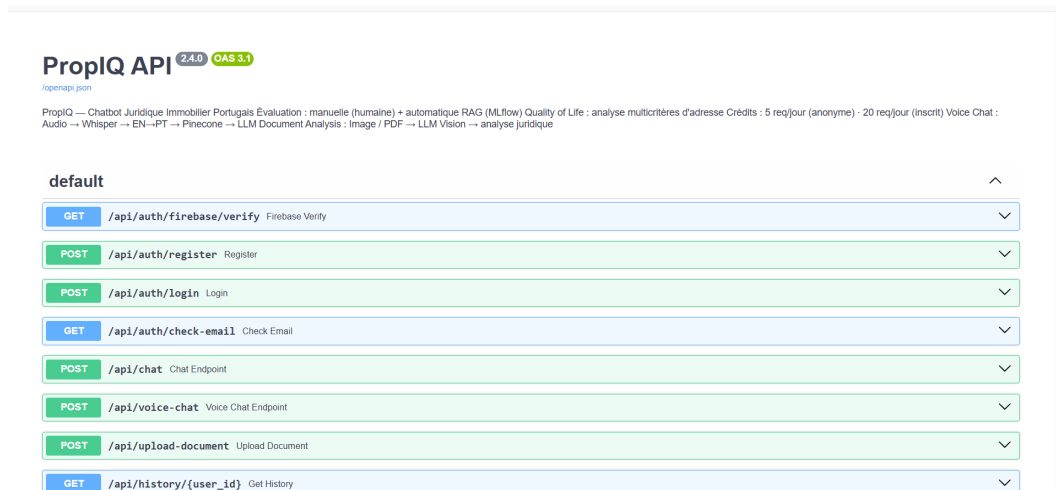


Figure 5.19: Documentation Swagger de l'API PropIQ

Authentification Le module d'authentification comprend notamment les routes suivantes :

- `/api/auth/firebase/verify` : vérification des jetons Firebase ;
- `/api/auth/register` : inscription d'un utilisateur ;
- `/api/auth/login` : connexion ;
- `/api/auth/check-email` : vérification de l'adresse e-mail.

Chatbot juridique et traitement multimodal La route principale du chatbot est `/api/chat`. Elle est complétée par deux routes permettant de traiter des entrées multimodales :

- `/api/voice-chat` : traitement des messages vocaux avec transcription via Whisper et intégration au pipeline RAG ;
- `/api/upload-document` : analyse des images et des fichiers PDF à l'aide d'un modèle de vision.

Historique des interactions L'historique des conversations est accessible via `/api/history/{user_id}`. Une interaction peut également être supprimée à l'aide de la route DELETE `/api/history/{history_id}`. La figure 5.20 présente les routes associées à l'historique, aux crédits et à l'analyse de qualité de vie.



DELETE	/api/history/{history_id}	Delete History Item
GET	/api/credits/status	Credits Status
POST	/api/credits/admin	Set Admin Credits
POST	/api/credits/reset	Reset User Credits
POST	/api/quality-life	Quality Life Alias
POST	/api/quality-of-life/analyze	Analyze Quality Of Life
GET	/api/quality-of-life/analyze	Analyze Quality Of Life Get
POST	/api/quality-of-life/geocode	Geocode
POST	/api/quality-of-life/pois	Get Pois
POST	/api/quality-of-life/air-quality	Get Air Quality
POST	/api/quality-of-life/score-only	Get Score Only
GET	/api/quality-of-life/config	Get Qol Config

Figure 5.20: Routes de gestion de l'historique, des crédits et de l'analyse de qualité de vie

Analyse de qualité de vie Le module de qualité de vie expose plusieurs routes :

- `/api/quality-of-life/analyze` : calcul du score de qualité de vie ;
- `/api/quality-of-life/geocode` : conversion d'une adresse en coordonnées ;
- `/api/quality-of-life/pois` : récupération des points d'intérêt à proximité ;
- `/api/quality-of-life/air-quality` : récupération des informations liées à la qualité de l'air.

Gestion des crédits La gestion des quotas est assurée par les routes `/api/credits/status`, `/api/credits/admin` et `/api/credits/reset`.

Avant l'exécution d'une fonctionnalité consommant un crédit, le système vérifie le solde disponible afin d'empêcher le dépassement du quota autorisé.

Évaluation et modèles Les fonctionnalités d'évaluation sont accessibles à travers plusieurs routes :

- `/api/evaluation/interaction/{interaction_id}` : consultation d'une interaction ;
- `/api/evaluation/submit/{interaction_id}` : enregistrement d'une évaluation manuelle ;

- `/api/rag-eval/run` : lancement d'une évaluation du pipeline RAG ;
- `/api/rag-eval/mlflow` : intégration de l'évaluation avec MLflow.

La route `/api/models` permet de récupérer la liste des modèles disponibles via Open-Router.

La figure 5.21 présente les routes associées à l'évaluation, aux modèles et aux abonnements.

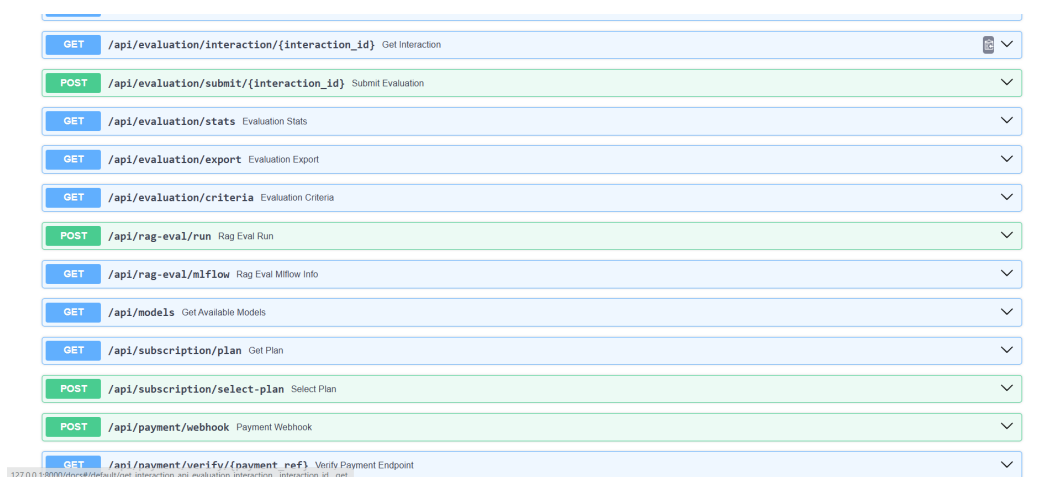


Figure 5.21: Routes d'évaluation, de modèles et de gestion des abonnements et paiements

Abonnements et paiement Les routes `/api/subscription/plan` et `/api/subscription/select-plan` permettent respectivement de consulter et de sélectionner un plan d'abonnement.

Les paiements sont ensuite traités à travers les routes `/api/payment/webhook` et `/api/payment/verify/{payment_ref}`, utilisées pour confirmer et vérifier les transactions.

5.7.2 Pipeline RAG et accès multi-LLM

Les documents juridiques préparés dans les chapitres précédents sont indexés dans Pinecone afin de permettre leur recherche sémantique.

Lorsqu'une question est reçue, le backend effectue la recherche des passages pertinents puis transmet le contexte récupéré au modèle de langage sélectionné via Open-Router. Cette architecture permet d'utiliser plusieurs modèles avec le même pipeline de recherche et de génération.

L'utilisation d'OpenRouter simplifie également la comparaison des modèles puisque le backend n'a pas besoin d'implémenter une intégration spécifique pour chaque fournisseur. L'API fournit notamment un endpoint permettant de lister les modèles disponibles et leurs caractéristiques [50].

5.7.3 Système de crédits et quotas

PropIQ distingue deux niveaux d'utilisation : une offre Free et une offre Premium. Chaque requête soumise aux fonctionnalités concernées consomme un crédit et le système vérifie le quota disponible avant de lancer le traitement.

L'offre **Free**, proposée à 0 €/mois, permet de découvrir les principales fonctionnalités de PropIQ avec des limites quotidiennes :

- 10 requêtes par jour auprès de l'assistant conversationnel ;
- 1 analyse d'image par jour ;
- 1 génération ou analyse de PDF par jour ;
- accès standard aux fonctionnalités de la plateforme.

L'offre **Premium**, proposée à 9,90 €/mois, augmente les limites disponibles et ajoute des fonctionnalités supplémentaires :

- 20 requêtes par jour auprès de l'assistant conversationnel ;
- 10 analyses d'images par jour ;
- 5 générations ou analyses de PDF par jour ;
- accès prioritaire au traitement ;
- support Premium.

Cette organisation permet de proposer une première utilisation gratuite tout en réservant des capacités supplémentaires aux utilisateurs disposant d'un abonnement.

L'intégration du paiement est réalisée avec Stripe et Konnect. La figure 5.22 présente le tableau de bord Stripe utilisé pour vérifier les transactions réalisées dans l'environnement de test.

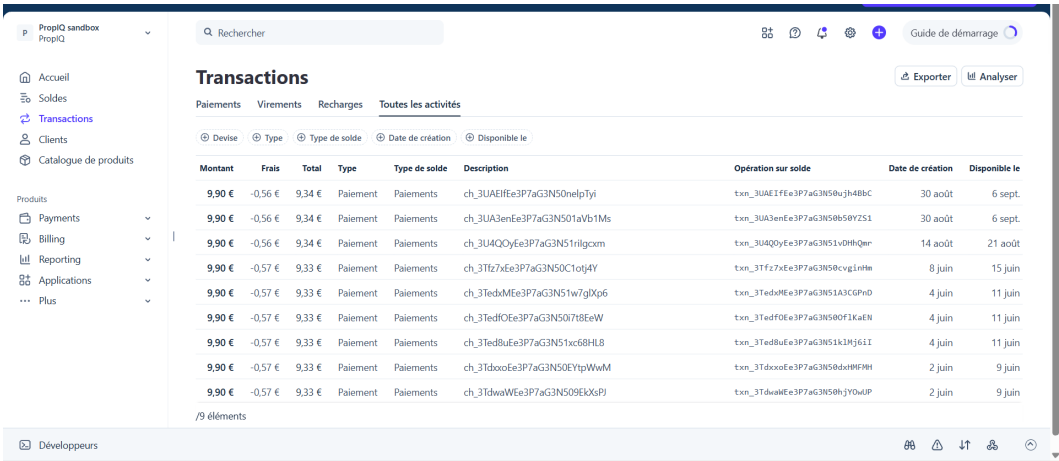


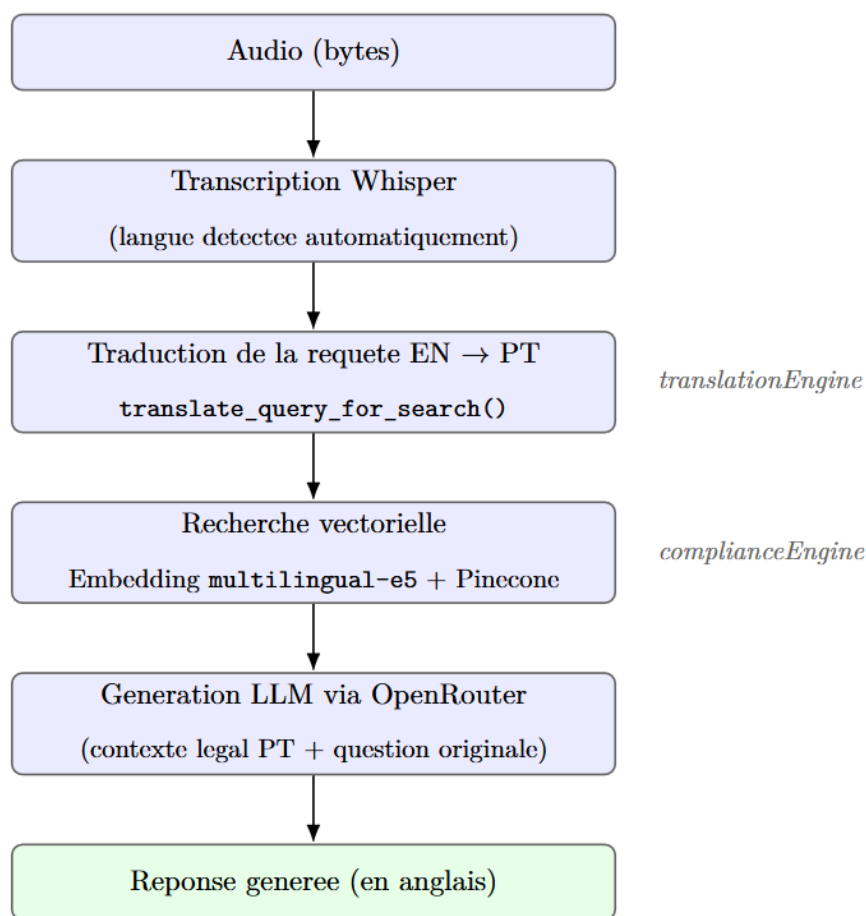
Figure 5.22: Tableau de bord Stripe pour les transactions des abonnements Premium

5.7.4 Transcription vocale

PropIQ permet à l'utilisateur d'interagir avec le chatbot à l'aide d'un message vocal. Le traitement commence par la transcription du message audio à l'aide de Whisper. La question obtenue est ensuite traitée par le pipeline linguistique. Elle peut être traduite vers le portugais afin d'améliorer la recherche dans le corpus juridique, principalement constitué de documents portugais. Les passages pertinents sont récupérés dans Pinecone puis transmis au modèle de langage.

La réponse générée est finalement retournée à l'utilisateur dans la langue attendue par l'application.

La figure 5.23 présente l'ensemble de ce processus.

**Figure 5.23:** Pipeline Voice-to-RAG

5.7.5 Authentification et sécurité

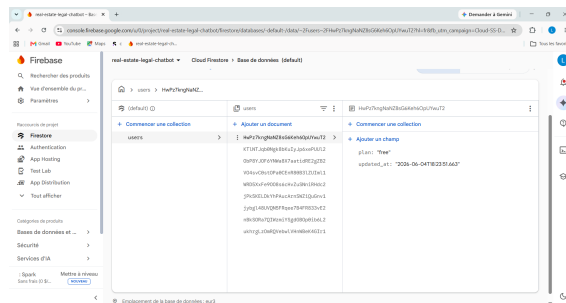
L'authentification de PropIQ repose sur Firebase Authentication. Sa mise en œuvre couvre la configuration du projet Firebase, l'intégration du SDK dans l'application Flutter, les parcours d'inscription et de connexion, la vérification des jetons côté FastAPI et la gestion des erreurs.

Deux modes de connexion sont disponibles : l'inscription avec une adresse e-mail et un mot de passe ainsi que Google Sign-In. Firebase Authentication prend en charge ces différents mécanismes et permet également d'utiliser les jetons d'authentification pour vérifier l'identité de l'utilisateur au niveau d'un backend personnalisé [15].

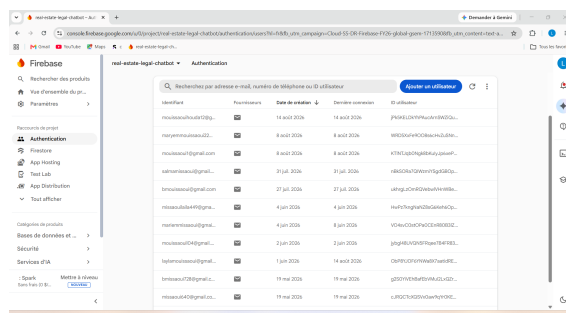
Le backend vérifie les jetons Firebase avant d'autoriser l'accès aux ressources protégées. Cette séparation permet de confier la gestion des informations d'authentification au service Firebase tout en conservant dans le backend la gestion des fonctionnalités propres

à PropIQ.

La figure 5.24 présente les interfaces Firebase utilisées pour superviser les utilisateurs et leurs métadonnées.



(a) Firestore



(b) Authentication

Figure 5.24: Console Firebase – gestion des utilisateurs et des métadonnées

5.7.6 Protection des données et aspects réglementaires

Le fonctionnement de PropIQ implique la manipulation de plusieurs catégories de données à caractère personnel : données d'identification et d'authentification gérées par Firebase, données de transaction traitées par Stripe et Konnect, ainsi que l'historique des échanges avec le chatbot juridique, susceptible de contenir des informations relatives à un bien immobilier ou à la situation de l'utilisateur.

La conception de PropIQ prend en compte plusieurs principes issus du Règlement Général sur la Protection des Données (RGPD), applicable notamment aux utilisateurs situés dans l'Union européenne. L'authentification et le stockage des identifiants sont délégués à Firebase Authentication, qui applique ses propres mécanismes de sécurité et de conformité [15]. Les données de paiement ne transitent pas directement par le backend de PropIQ : elles sont traitées par Stripe et Konnect, ce qui limite

la responsabilité directe de l'application dans la gestion des informations bancaires. L'historique des interactions est quant à lui stocké localement dans SQLite et reste accessible uniquement à l'utilisateur authentifié concerné, via son identifiant.

Au-delà des aspects techniques de protection des données, PropIQ soulève également une question éthique liée à la nature juridique de son contenu. Le chatbot est conçu pour fournir des informations à partir de textes réglementaires portugais, mais il ne constitue pas un substitut à un avis juridique professionnel. Cette limite est explicitement rappelée à l'utilisateur : lorsque les documents récupérés ne permettent pas de répondre avec un niveau de confiance suffisant, l'application invite l'utilisateur à consulter une source officielle ou un professionnel de l'immobilier certifié plutôt que de formuler une réponse incertaine (cf. section 5.4.3 et le mécanisme de niveau de confiance décrit au chapitre 4). Cette approche vise à limiter le risque qu'un utilisateur se fonde exclusivement sur une réponse générée automatiquement pour une décision juridique ou financière importante, sans toutefois se substituer à un encadrement réglementaire formel du statut d'un tel assistant, qui resterait à approfondir dans une phase ultérieure du projet.

5.8 Interfaces graphiques de l'application

5.8.1 Application mobile Flutter

L'application mobile constitue l'interface principale de PropIQ. Elle permet à l'utilisateur de parcourir les différentes fonctionnalités depuis un appareil mobile.

Le parcours commence par l'écran de bienvenue, suivi de l'inscription ou de la connexion. L'utilisateur peut ensuite sélectionner son offre et accéder aux fonctionnalités de l'application.

Le paiement Premium est intégré à travers un parcours comprenant notamment la gestion des redirections par *deep link*. Une fois l'utilisateur authentifié, il accède au tableau de bord, au chatbot juridique et aux fonctionnalités complémentaires telles que l'analyse documentaire et l'analyse de qualité de vie.

5.8.1.1 Refonte visuelle

L'interface de PropIQ a été améliorée progressivement au cours du développement. Une première version utilisait principalement une identité visuelle basée sur des tons violets. Cette version a ensuite été remplacée par une identité graphique dominée par le bleu foncé, appliquée de manière cohérente aux différents écrans.

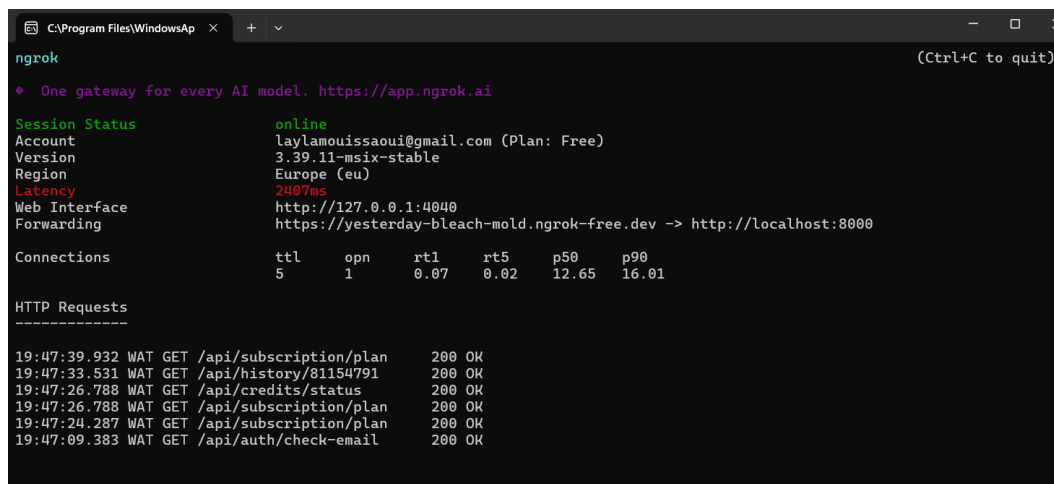
Cette évolution visuelle avait pour objectif d'obtenir une interface plus homogène et adaptée au positionnement de l'application.

5.8.1.2 Gestion des dépendances et tests sur appareil physique

La mise à niveau de l'environnement Flutter a nécessité l'adaptation de plusieurs dépendances afin de garantir leur compatibilité.

Des tests ont également été réalisés sur un appareil physique. Durant ces tests, Ngrok a permis d'exposer temporairement l'API FastAPI exécutée localement afin que l'application mobile puisse communiquer avec le backend [49].

La figure 5.25 montre le tunnel Ngrok utilisé pour cette communication.



```

ngrok                                     (Ctrl+C to quit)
♦ One gateway for every AI model. https://app.ngrok.ai

Session Status      online
Account             laylamouissaoui@gmail.com (Plan: Free)
Version             3.39.11-msix-stable
Region              Europe (eu)
Latency             2407ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://yesterday-bleach-mold.ngrok-free.dev -> http://localhost:8000

Connections      ttl    opn    rt1    rt5    p50    p90
                  5      1      0.07   0.02   12.65   16.01

HTTP Requests
-----
19:47:39.932 WAT GET /api/subscription/plan 200 OK
19:47:33.531 WAT GET /api/history/81154791 200 OK
19:47:26.788 WAT GET /api/credits/status 200 OK
19:47:26.788 WAT GET /api/subscription/plan 200 OK
19:47:24.287 WAT GET /api/subscription/plan 200 OK
19:47:09.383 WAT GET /api/auth/check-email 200 OK

```

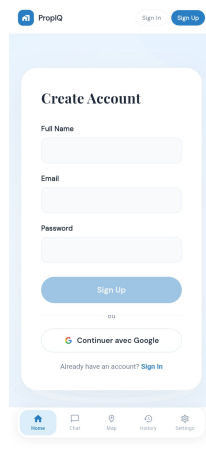
Figure 5.25: Exposition temporaire de l'API locale avec Ngrok

5.8.1.3 Captures d'écran de l'application

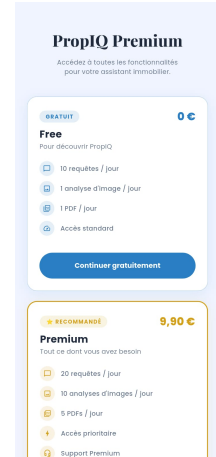
Les figures suivantes présentent les principales interfaces de l'application mobile PropIQ.



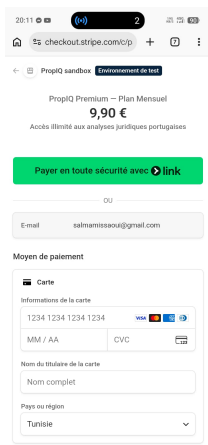
(a) Page de bienvenue



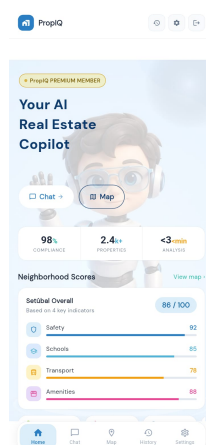
(b) Inscription



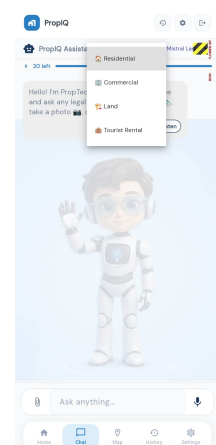
(c) Choix du plan



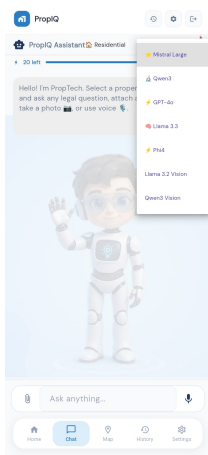
(d) Paiement



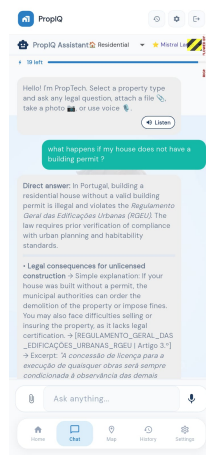
(e) Accueil



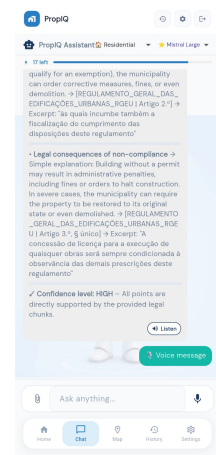
(f) Choix du type de bien



(g) Choix du modèle IA

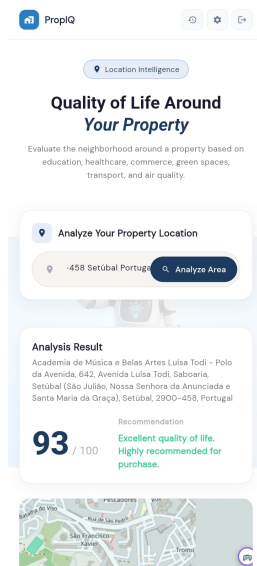


(h) Réponse du chatbot

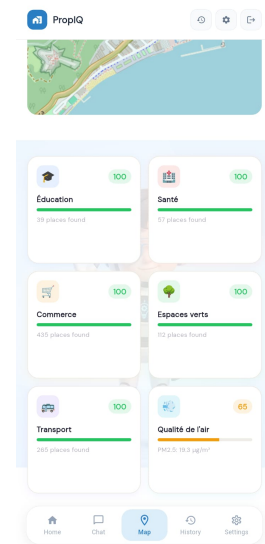


(i) Question par vocal

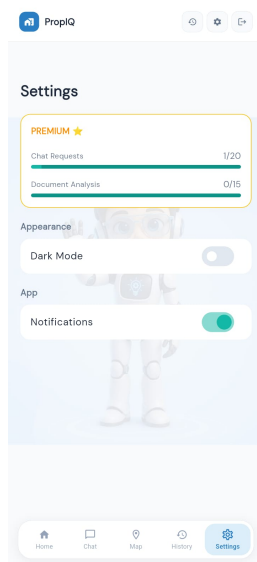




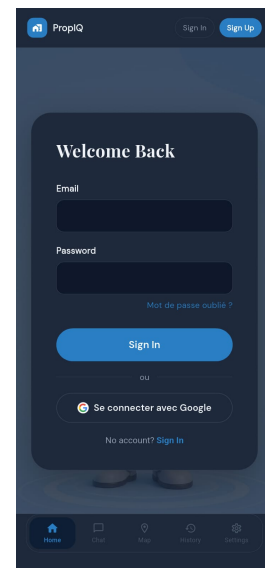
(a) Analyse de qualité de vie (1)



(b) Analyse de qualité de vie (2)



(c) Paramètres



(d) Mode sombre

Figure 5.27: Captures d'écran de l'application mobile PropIQ – qualité de vie et paramètres (2/2)

5.9 Conclusion

Ce chapitre a présenté la mise en œuvre et le déploiement de la solution PropIQ. Les principaux choix technologiques ont été détaillés, notamment l'utilisation de Flutter pour l'application mobile, FastAPI pour le backend, Pinecone pour la recherche vectorielle et OpenRouter pour l'accès aux modèles de langage.

La migration de FAISS vers Pinecone a également été présentée comme une évolution visant à simplifier la gestion de l'index vectoriel et à mieux répondre aux besoins du pipeline RAG.

Les différents diagrammes ont ensuite permis de décrire la structure fonctionnelle et technique de la solution. L'architecture globale, le pipeline NLP/RAG et la chaîne MLOps montrent comment les différents composants sont intégrés au sein de PropIQ.

La réalisation du backend a permis d'intégrer les principales fonctionnalités de l'application : chatbot juridique, analyse documentaire, interaction vocale, analyse de qualité de vie, authentification, historique, quotas et paiement. Enfin, les interfaces Flutter illustrent le résultat fonctionnel obtenu.

Le chapitre suivant est consacré aux tests et à la validation de PropIQ. Il permettra de vérifier le comportement de la solution à travers différents scénarios et de mesurer son adéquation avec les objectifs définis au début du projet.

CHAPITRE 6

Tests et validation

6.1 Introduction

Ce chapitre présente la stratégie de validation mise en place pour évaluer la solution PropIQ, les résultats obtenus sur ses différents modules, ainsi qu’une discussion critique portant sur les limites et les perspectives d’évolution du projet. La démarche combine des tests de niveau code (tests unitaires, tests boîte blanche) et des tests de niveau système (tests boîte noire, tests d’intégration, tests de charge, tests de sécurité), complétés par une évaluation spécifique des composants à base d’intelligence artificielle (chatbot juridique RAG, indice de qualité de vie) et par une supervision continue en production.

6.2 Plan de validation

Conformément à la phase *Evaluation* de la méthodologie CRISP-DM, la validation du système a porté sur plusieurs axes complémentaires :

- des **tests unitaires** vérifiant le comportement isolé des fonctions et endpoints critiques du backend FastAPI ;
- des **tests boîte blanche**, fondés sur la connaissance du code source, visant à couvrir les différents chemins logiques (gestion des erreurs, cas limites, bascule entre fournisseurs de paiement) ;

- des **tests boîte noire**, exécutés sans connaissance de l'implémentation, vérifiant le comportement fonctionnel de l'application du point de vue de l'utilisateur final (application Flutter) ;
- des **tests de charge** évaluant la latence et la stabilité de l'API sous plusieurs requêtes simultanées ;
- des **tests de sécurité** automatisés (analyse de secrets, dépendances) intégrés au pipeline CI/CD ;
- la **pertinence du chatbot juridique** fondé sur le RAG, évaluée par comparaison objective entre plusieurs modèles de langage via un suivi MLflow ;
- la **robustesse de l'indice de qualité de vie**, évaluée par cohérence avec les caractéristiques géographiques et environnementales observables dans des zones connues ;
- une **supervision continue en production**, basée sur la stack Elasticsearch/Kibana, assurant un suivi post-déploiement de la latence et du taux d'erreur de l'API.

6.3 Tests unitaires

Les tests unitaires ciblent les fonctions et endpoints les plus critiques du backend FastAPI, notamment la logique métier indépendante des services externes (calcul du quota utilisateur, sélection du fournisseur de paiement, formatage des réponses du chatbot). Ils ont été écrits avec `pytest` et intégrés au pipeline GitHub Actions, avec un seuil de couverture de code fixé à 80 %.

Ce type de test a notamment permis de détecter et de corriger le bug où l'endpoint de vérification de paiement appelait le mauvais fournisseur selon le préfixe de `payment_ref` (cf. section 6.9).

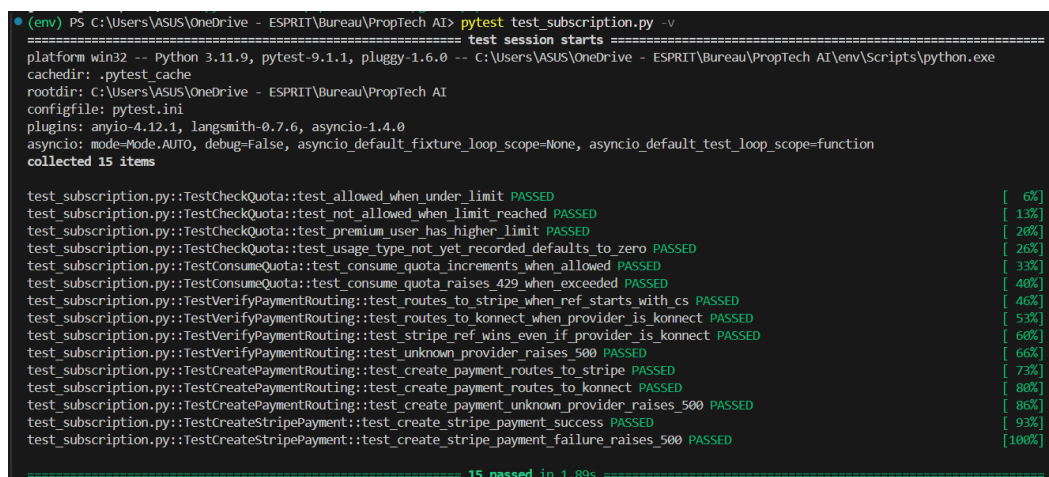


Figure 6.1: Rapport de couverture de code généré par pytest-cov (seuil de 80 % atteint)

6.4 Tests boîte blanche

Les tests boîte blanche s'appuient sur la connaissance du code source pour vérifier que l'ensemble des chemins logiques significatifs sont couverts, en particulier :

- les chemins de vérification du token d'authentification Firebase (SDK non initialisé, en-tête d'autorisation absent ou mal formé, token vide, expiré, invalide, révoqué, ou valide) ;
- la validation de l'adresse email lors de l'inscription (format invalide, domaine inexistant via vérification DNS, email déjà enregistré, email valide et disponible) ;
- les branches conditionnelles de la sélection de plan (plan invalide, activation immédiate du plan gratuit, génération du lien de paiement pour le plan premium) ;
- le traitement du webhook de paiement (champs manquants ignorés, paiement complété activant le premium, paiement en attente non activé, exception gérée avec retour d'un statut d'erreur) ;
- la validation des données d'entrée lors de la préparation d'une demande (type de propriété invalide).

```

PS C:\Users\ASUS\OneDrive - ESPRIT\Bureau\PropTech AI> pytest test_boite_blanche.py -v
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\ASUS\OneDrive - ESPRIT\Bureau\PropTech AI\env\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ASUS\OneDrive - ESPRIT\Bureau\PropTech AI
configfile: pytest.ini
plugins: anyio-4.12.1, langsmith-0.7.6, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 22 items

test_boite_blanche.py::TestVerifyFirebaseToken::test_firebase_not_initialized_raises_503 PASSED [ 45%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_missing_authorization_header_raises_401 PASSED [ 50%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_header_without_bearer_prefix_raises_401 PASSED [ 55%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_empty_token_raises_401 PASSED [ 60%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_expired_token_raises_401 PASSED [ 65%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_invalid_token_raises_401 PASSED [ 70%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_revoked_token_raises_401 PASSED [ 75%]
test_boite_blanche.py::TestVerifyFirebaseToken::test_valid_token_returns_decoded_payload PASSED [ 80%]
test_boite_blanche.py::TestCheckEmail::test_domain_exists_true_on_successful_dns PASSED [ 85%]
test_boite_blanche.py::TestCheckEmail::test_domain_exists_false_on_dns_error PASSED [ 90%]
test_boite_blanche.py::TestCheckEmail::test_check_email_invalid_format PASSED [ 95%]
test_boite_blanche.py::TestCheckEmail::test_check_email_domain_does_not_exist PASSED [ 100%]
test_boite_blanche.py::TestCheckEmail::test_check_email_already_registered PASSED [ 100%]
test_boite_blanche.py::TestCheckEmail::test_check_email_valid_and_available PASSED [ 100%]
test_boite_blanche.py::TestSelectPlan::test_invalid_plan_raises_400 PASSED [ 100%]
test_boite_blanche.py::TestSelectPlan::test_free_plan_activates_immediately PASSED [ 100%]
test_boite_blanche.py::TestSelectPlan::test_premium_plan_returns_payment_link PASSED [ 100%]
test_boite_blanche.py::TestPaymentWebhook::test_missing_fields_returns_ignored PASSED [ 100%]
test_boite_blanche.py::TestPaymentWebhook::test_completed_payment_activates_premium PASSED [ 100%]
test_boite_blanche.py::TestPaymentWebhook::test_pending_payment_not_activated PASSED [ 100%]
test_boite_blanche.py::TestPaymentWebhook::test_exception_during_webhook_returns_error_status PASSED [ 100%]
test_boite_blanche.py::TestPrepareDataValidation::test_invalid_property_type_raises_value_error PASSED [ 100%]

===== warnings summary =====

```

Figure 6.2: Résultat de l'exécution des tests boîte blanche (pytest -v, 22 tests)

Le tableau 6.1 synthétise les principaux cas couverts, sur un total de 22 tests tous validés avec succès.

Table 6.1: Synthèse des tests boîte blanche

Module	Chemin testé	Résultat
Auth	Token expiré, invalide ou révoqué	Erreur 401 correctement levée
Auth	Token valide	Payload décodé et retourné
Email	Domaine inexistant (échec DNS)	Email refusé comme invalide
Plan	Plan invalide demandé	Erreur 400 correctement levée
Paieement	Webhook avec paiement complété	Activation automatique du premium
Paieement	Exception pendant le traitement du webhook	Statut d'erreur retourné proprement

6.5 Tests boîte noire

Les tests boîte noire ont été réalisés sans connaissance de l'implémentation interne, en manipulant directement l'application Flutter afin de valider le comportement attendu du point de vue utilisateur : envoi d'un message au chatbot, téléversement d'un document PDF, souscription à un plan Premium, consultation de l'indice de qualité de vie d'une zone. Chaque scénario a été documenté par une capture d'écran de l'interface avant et après action.

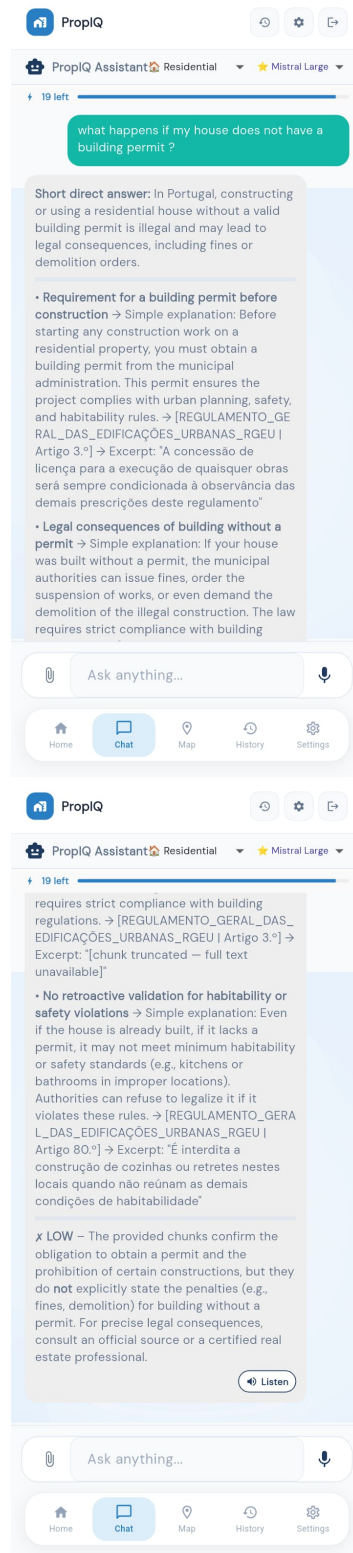


Figure 6.3: Test boîte noire : envoi d’une question juridique et réponse du chatbot dans l’application Flutter

6.6 Tests de charge et de performance

Des tests de charge ont été effectués sur l'API FastAPI afin d'évaluer sa stabilité sous des requêtes simultanées vers le chatbot juridique et le module d'indice de qualité de vie. L'objectif était de mesurer la capacité du backend à maintenir un temps de réponse acceptable tout en limitant les erreurs lors de sollicitations concurrentes.

Les principaux indicateurs suivis sont le nombre de requêtes simultanées, le nombre total de requêtes exécutées, la durée du test, le temps de réponse moyen, le temps de réponse maximal, ainsi que le taux d'erreur. Lorsque les données sont disponibles, le percentile 95 (p95) du temps de réponse est également considéré afin de mieux caractériser les requêtes les plus lentes.

Les tests ont été réalisés dans l'environnement d'exécution utilisé pour le développement et la validation de PropIQ. Les résultats obtenus ont ensuite été observés à l'aide de la stack Elasticsearch/Kibana afin de vérifier la latence des requêtes et les éventuelles erreurs générées par l'API.

Table 6.2: Paramètres et indicateurs des tests de charge

Indicateur	Valeur mesurée
Nombre d'utilisateurs simultanés	1 utilisateur virtuel (Locust)
Nombre total de requêtes	23 requêtes (dont 3 échecs)
Débit moyen	0,3 requête/seconde
Temps de réponse médian	1800 ms
Temps de réponse moyen	2242,91 ms
Temps de réponse minimal	3 ms
Temps de réponse maximal	10272 ms
Temps de réponse p95	4900 ms
Temps de réponse p99	10000 ms
Taux d'erreur	13 % (3 échecs sur 23 requêtes)
Environnement d'exécution	Instance locale (<code>localhost:8000</code>), test réalisé avec l'outil Locust

Les résultats de ces mesures doivent être interprétés conjointement avec les journaux de supervision. Une faible latence moyenne ne suffit pas à caractériser la performance globale du système ; l'analyse du p95, lorsqu'il est disponible, permet notamment d'identifier les dégradations observées sur les requêtes les plus lentes.

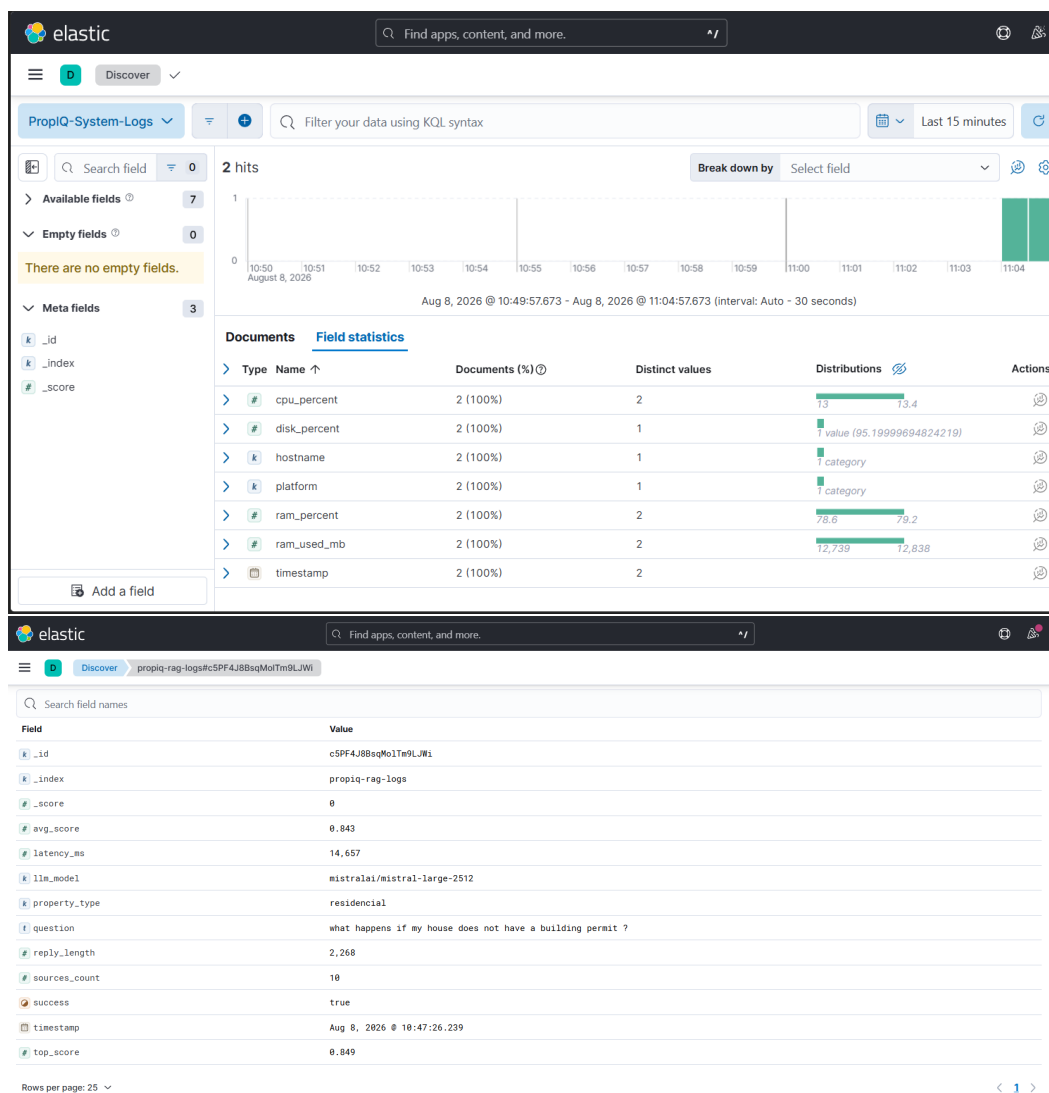
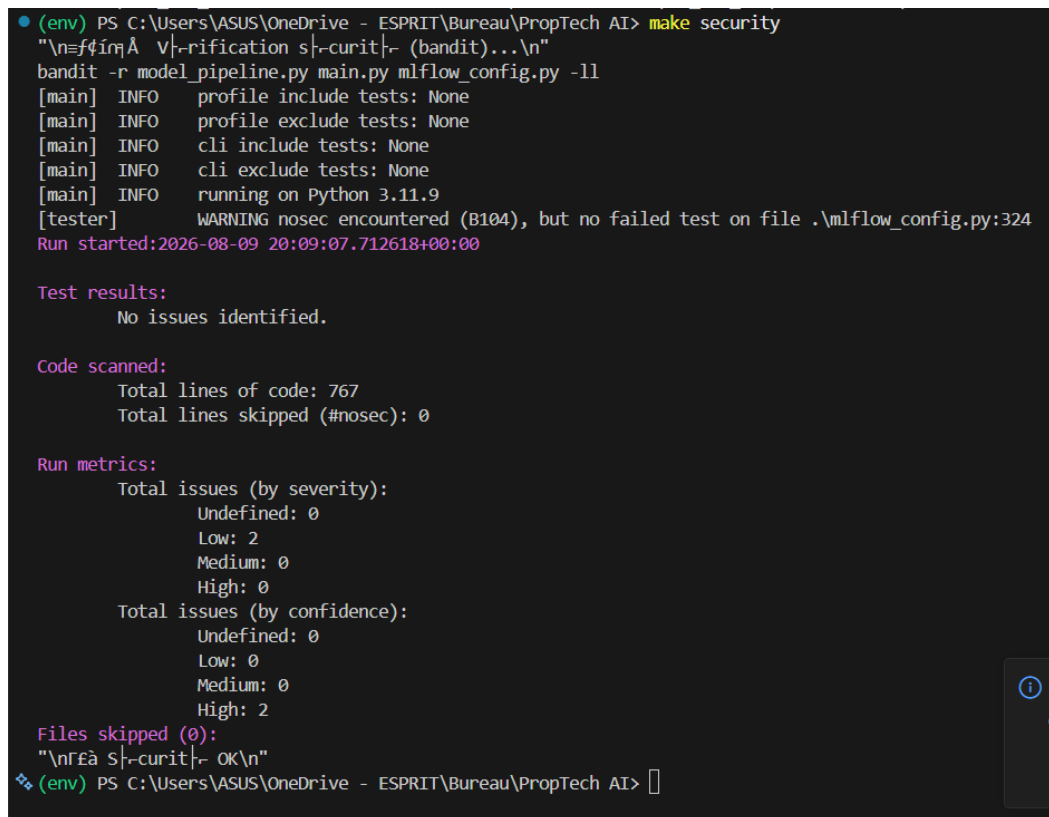


Figure 6.4: Tableau de bord Kibana : latence et taux d'erreur de l'API en conditions de charge

6.7 Tests de sécurité

En complément du scan de secrets, une analyse statique du code Python a été intégrée via l'outil `bandit` (cible `make security` du Makefile), exécutée sur les trois modules critiques du backend (`model_pipeline.py`, `main.py`, `mlflow_config.py`), représentant 767 lignes de code analysées. Le rapport ne signale aucune vulnérabilité de sévérité Medium ou High, seuil retenu pour valider l'étape (*Securite OK*). Deux observations de sévérité Low et de confiance élevée ont néanmoins été relevées dans les métriques du scan, sans faire échouer la pipeline car situées en dessous du seuil de blocage configuré (-11). Une exception `#nosec` présente dans `mlflow_config.py` (ligne 324, règle B104

relative à une liaison réseau sur toutes les interfaces) a par ailleurs été vérifiée par bandit sans générer d'échec de test durant ce run.



```

(env) PS C:\Users\ASUS\OneDrive - ESPRIT\Bureau\PropTech AI> make security
"\n=fiqÅ V|r-rification s|r-curit|r- (bandit)...\n"
bandit -r model_pipeline.py main.py mlflow_config.py -ll
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.11.9
[tester] WARNING nosecc encountered (B104), but no failed test on file .\mlflow_config.py:324
Run started:2026-08-09 20:09:07.712618+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 767
    Total lines skipped (#nosec): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 2
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 2
Files skipped (0):
"\nrfÅ s|r-curit|r- OK\n"
(env) PS C:\Users\ASUS\OneDrive - ESPRIT\Bureau\PropTech AI>

```

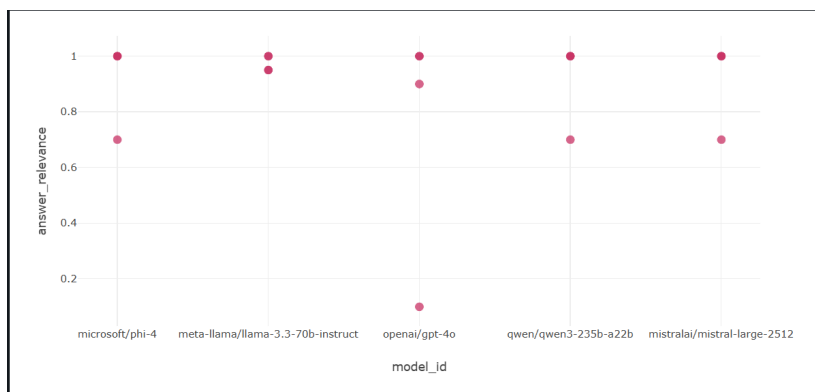
Figure 6.5: Résultat de l'analyse statique de sécurité avec bandit (make security)

Au niveau applicatif, la sécurité est également assurée par l'authentification Firebase (email/mot de passe et Google Sign-In), avec vérification du token d'identification Firebase (ID token) à chaque requête adressée aux endpoints protégés de l'API, ainsi que par un contrôle d'accès basé sur des quotas différenciés selon le plan de l'utilisateur (Free : 10 requêtes/jour, Premium : 20 requêtes/jour), limitant les risques d'usurpation de session et d'usage non autorisé du service.

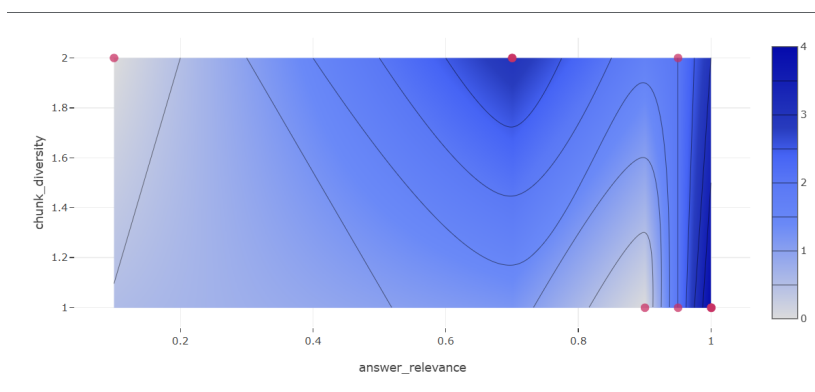
6.8 Protocoles et conditions expérimentales des modules IA

L'évaluation du chatbot juridique a été menée sur un ensemble de scénarios représentatifs (bien résidentiel en zone urbaine, terrain à usage commercial, bien avec statut administratif incertain, alojamiento local), en confrontant les réponses générées à celles

attendues d'un point de vue juridique. Le suivi MLflow, étendu à **cinq modèles de langage** disponibles via OpenRouter (Qwen3-235B-A22B, Mistral Large 3, GPT-4o, Llama 3.3 70B, Phi-4), a permis de comparer objectivement la pertinence des réponses et la diversité des sources juridiques mobilisées par chaque modèle, afin de sélectionner la configuration la plus adaptée à la production.



(a) Pertinence (`answer_relevance`) par modèle, un point par scénario évalué



(b) Pertinence en fonction de la diversité des sources mobilisées (`chunk_diversity`)

Figure 6.6: Interface MLflow : comparaison des cinq modèles LLM sur la pertinence des réponses et la diversité des sources juridiques mobilisées

La figure 6.6a montre que la majorité des réponses obtiennent un score de pertinence élevé (0.9 à 1.0) sur la plupart des scénarios testés, chaque modèle présentant néanmoins une réponse plus faible correspondant généralement au scénario le plus complexe. Le cas le plus critique est celui de GPT-4o, dont la pertinence chute à environ 0.1 sur le scénario commercial, correspondant à un refus de réponse malgré un contexte juridique disponible.

La figure 6.6b met en évidence que la majorité des réponses pertinentes mobilisent

une seule catégorie légale (`chunk_diversity = 1`), tandis que les rares cas mobilisant deux catégories distinctes présentent des résultats plus contrastés, suggérant qu’une diversification du contexte récupéré ne garantit pas systématiquement une meilleure pertinence et peut au contraire signaler une confusion du module de retrieval sur les scénarios ambigus.

La pertinence de l’indice de qualité de vie a été évaluée en confrontant les scores générés par le système à une vérification qualitative de zones connues de la région de Setúbal. Cette vérification a porté sur la cohérence entre les scores obtenus et les caractéristiques géographiques et environnementales observables, notamment la présence de transports, d’établissements scolaires, de structures de santé, de commerces et d’espaces verts. Les données utilisées pour cette analyse proviennent des sources décrites dans les chapitres précédents, notamment OpenStreetMap/Overpass pour les points d’intérêt et Open-Meteo pour les données de qualité de l’air.

Enfin, le middleware de monitoring FastAPI, associé à la stack Elasticsearch/Kibana, a permis de suivre en continu la latence des requêtes et le taux d’erreur de l’API en conditions réelles, avec des alertes automatiques transmises via Slack en cas de dépassement de seuil.

6.9 Discussion critique

Plusieurs limites doivent être soulignées. Premièrement, la qualité des réponses du chatbot juridique reste dépendante de l’exhaustivité et de la mise à jour du corpus juridique indexé dans Pinecone ; toute évolution réglementaire non intégrée rapidement peut dégrader la pertinence des réponses. Deuxièmement, l’indice de qualité de vie repose sur des données géographiques et environnementales publiques dont la représentativité peut varier selon les zones géographiques, certaines localités disposant de peu de données exploitables. Troisièmement, le recours à des modèles de langage tiers via OpenRouter introduit une dépendance à la disponibilité et à l’évolution tarifaire de ces fournisseurs. Enfin, les tests boîte blanche et boîte noire actuels, bien que couvrant les scénarios critiques identifiés, ne constituent pas une couverture exhaustive de l’ensemble des cas d’usage possibles de l’application.

6.10 Perspectives

Les perspectives d'évolution identifiées incluent la mise à jour automatique de la base de connaissances juridique à mesure de la publication de nouveaux textes, l'extension du périmètre géographique au-delà de Setúbal, l'ajout de notifications intelligentes en cas de changement réglementaire affectant un bien déjà audité, l'élargissement des capacités du chatbot afin qu'il puisse répondre à toute question juridique en se basant sur les lois d'autres pays au-delà du cadre portugais, ainsi que la génération automatisée de rapports exportables au format PDF destinés aux clients et aux experts. Sur le plan des tests, une extension de la couverture des tests unitaires et l'ajout de tests de charge automatisés dans le pipeline CI/CD sont également envisagés.

6.11 Conclusion

Ce chapitre a présenté la démarche de validation de la solution PropIQ, structurée autour de tests unitaires, de tests boîte blanche et boîte noire, de tests d'intégration, de charge et de sécurité, ainsi que d'une évaluation spécifique des composants d'intelligence artificielle. Les principaux résultats obtenus sur les différents modules ont été présentés et illustrés par des captures d'écran des outils de test et de supervision.

Les tests de performance ont permis d'étudier le comportement de l'API FastAPI sous des sollicitations simultanées en suivant notamment la latence et le taux d'erreur. Les résultats doivent être interprétés à partir du nombre de requêtes exécutées, de la durée des scénarios et des métriques de temps de réponse disponibles, afin de caractériser de manière objective les performances du système.

La discussion menée met en évidence des limites maîtrisées et ouvre sur des perspectives d'évolution concrètes, développées dans la conclusion générale de ce rapport.

Conclusion générale

Ce projet de fin d'études, réalisé au sein de Remax Wise (Portugal), avait pour objectif de concevoir un **auditeur réglementaire intelligent** destiné au secteur **immobilier** portugais, capable d'automatiser la vérification de **conformité légale** d'un bien au regard du cadre **Simplex** et des règlements municipaux de Setúbal, tout en apportant une vision complémentaire de la **qualité de vie** environnante fondée sur l'analyse de données textuelles et géographiques publiques. Ce dernier chapitre revient sur le cheminement suivi tout au long du rapport, avant de dégager les principaux apports du projet et ses perspectives d'évolution.

Le point de départ du projet, présenté au chapitre 1, a consisté à situer la solution dans son contexte institutionnel et réglementaire. L'étude de l'existant a mis en évidence l'absence, sur le marché, d'un outil combinant à la fois une spécialisation sur le cadre Simplex et les règlements municipaux, un chatbot juridique explicable fondé sur le RAG, et un indicateur de qualité de vie – justifiant ainsi le positionnement retenu pour PropIQ face aux solutions LegalTech généralistes et aux plateformes immobilières existantes.

Le chapitre 2 a ensuite formalisé les besoins fonctionnels et non fonctionnels du système ainsi que le cadre méthodologique du projet, structuré selon la méthodologie **CRISP-DM** afin d'organiser l'ensemble du cycle de vie data science, de la compréhension métier jusqu'au déploiement, depuis la constitution du corpus juridique initial jusqu'à la mise en place des abonnements et de la supervision en production.

Sur le plan de la conception et du déploiement, détaillés au chapitre 5, plusieurs choix technologiques ont été affinés au fil du projet – notamment la migration de FAISS vers **Pinecone** pour l'indexation vectorielle (cf. section 5.3), motivée par une comparaison qualitative ayant mis en évidence les avantages de Pinecone en matière de simplicité

d'exploitation, de temps de réponse perçu et de fiabilité de service. L'architecture retenue s'appuie sur une application mobile **Flutter**, une API backend **FastAPI**, et un accès multi-LLM via **OpenRouter** donnant accès à sept modèles de langage complémentaires, dont les architectures variées (denses, à mixture d'experts, textuelles ou vision-langage) permettent de couvrir aussi bien le raisonnement juridique pur que l'analyse multimodale des pièces jointes.

Ce même chapitre décrit également la mise en œuvre technique qui a concrétisé cette architecture à travers un backend FastAPI structuré par domaine fonctionnel, un pipeline RAG couplé à un système de crédits et de quotas, une intégration de **Whisper** pour l'interaction vocale, ainsi qu'une couche **MLOps** complète déployée en sept phases – de la modularisation du code jusqu'à la supervision applicative via MLflow et la stack Elasticsearch/Kibana, en passant par la conteneurisation Docker et l'automatisation CI/CD avec GitHub Actions. Le développement de l'application mobile Flutter a par ailleurs suivi une démarche DevSecOps, incluant analyse statique et lint automatisés.

Enfin, la démarche de validation présentée au chapitre 6 a confirmé la capacité du système à fournir des réponses juridiques pertinentes et justifiées, avec un score de pertinence élevé sur la majorité des scénarios testés parmi les cinq modèles comparés via MLflow, ainsi qu'un indicateur de qualité de vie cohérent avec les réalités locales observées. Cette phase a également permis d'identifier des limites concrètes – dépendance à la mise à jour du corpus juridique, variabilité de la représentativité des données selon les zones géographiques, dépendance aux fournisseurs de modèles tiers – tout en confirmant le respect des exigences de **sécurité**, de **scalabilité** et de **traçabilité** attendues d'une solution LegalTech, via des tests unitaires, boîte blanche, boîte noire, de charge et de sécurité intégrés au pipeline CI/CD.

Au-delà des résultats techniques, ce projet a constitué une expérience formatrice sur le plan de la conduite d'un projet data science de bout en bout, de l'articulation entre exigences juridiques et contraintes d'ingénierie logicielle, et de la collaboration avec un partenaire professionnel du secteur immobilier. Les perspectives identifiées au chapitre 6 – mise à jour continue de la base de connaissances juridique, extension géographique au-delà de Setúbal, notifications réglementaires automatiques, élargissement du chatbot à d'autres cadres juridiques nationaux, et génération de rapports exportables – tracent une feuille de route claire pour les évolutions futures de la plateforme

PropIQ.

Bibliography

- [1] IBM. (2026) Ibm spss modeler crisp-dm guide. Documentation officielle IBM (PDF). [Online]. Available: <https://public.dhe.ibm.com/software/analytics/spss/documentation/modeler/18.2/en/ModelerCRISPDM.pdf>
- [2] Agência para a Reforma Tecnológica do Estado (ARTE). (2026) Arte – agência para a reforma tecnológica do estado. [Online]. Available: <https://www.arte.gov.pt/>
- [3] Governo de Portugal. (2024) Decreto-lei n.º 10/2024, de 8 de janeiro. Consultar via motor de pesquisa do Diário da República Eletrónico – URL específico não confirmado. [Online]. Available: <https://diariodarepublica.pt/dr/home>
- [4] Câmara Municipal de Setúbal. (2026) Plano diretor municipal de setúbal. [Online]. Available: <https://www.mun-setubal.pt/atividade-municipal/urbanismo/pdm/>
- [5] ——. (2026) Urbanismo – município de setúbal. [Online]. Available: <https://www.mun-setubal.pt/atividade-municipal/urbanismo/>
- [6] LexisNexis. (2026) Lexis+. [Online]. Available: <https://www.lexisnexis.com/en-int/products/lexis-plus>
- [7] Thomson Reuters. (2026) Westlaw. [Online]. Available: <https://legal.thomsonreuters.com/en/products/westlaw>
- [8] ——. (2026) About cocounsel. [Online]. Available: <https://www.thomsonreuters.com/en-us/help/cocounsel/legal/get-started/about>
- [9] idealista. (2026) idealista – portal imobiliário em portugal. L’URL d’origine (/comprar-casas/) renvoyait une erreur 404 ; structure d’URL à revérifier pour un lien plus spécifique. [Online]. Available: <https://www.idealista.pt/>

- [10] Imovirtual. (2026) Imovirtual – portal imobiliário. [Online]. Available: <https://www.imovirtual.com/>
- [11] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, “Hallucination-free? assessing the reliability of leading AI legal research tools,” *Journal of Empirical Legal Studies*, vol. 22, no. 2, pp. 216–242, 2025. [Online]. Available: <https://doi.org/10.1111/jels.12413>
- [12] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, “Large legal fictions: Profiling legal hallucinations in large language models,” *Journal of Legal Analysis*, vol. 16, no. 1, pp. 64–93, 2024. [Online]. Available: <https://arxiv.org/abs/2401.01301>
- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [14] OpenRouter. (2026) Openrouter documentation: Quickstart. [Online]. Available: <https://openrouter.ai/docs/quickstart>
- [15] Google. (2026) Firebase authentication documentation. [Online]. Available: <https://firebase.google.com/docs/auth>
- [16] Agile Business Consortium. (2026) Moscow prioritisation. [Online]. Available: https://www.agilebusiness.org/page/ProjectFramework_10_MoSCoWPrioritisation
- [17] OpenStreetMap Foundation. (2026) Overpass api. Documentation OpenStreetMap. [Online]. Available: https://wiki.openstreetmap.org/wiki/Overpass_API
- [18] Open-Meteo. (2026) Open-meteo air quality api. Documentation officielle de l’API Open-Meteo Air Quality. [Online]. Available: <https://open-meteo.com/en/docs/air-quality-api>

- [19] Google. (2026) Geocoding api documentation. [Online]. Available: <https://developers.google.com/maps/documentation/geocoding/overview>
- [20] IBM. (2025) Understanding and preparing data. Documentation officielle IBM. [Online]. Available: <https://www.ibm.com/docs/en/ws-and-kc?topic=modeler-understanding-preparing-data>
- [21] Diário da República. (2026) Diário da república eletrónico. Portail officiel de la República Portuguesa (URL corrigée : redirection automatique vers /dr/home). [Online]. Available: <https://diariodarepublica.pt/dr/home>
- [22] J. Singer-Vine. (2026) pdfplumber. Dépôt et documentation du projet. [Online]. Available: <https://github.com/jsvine/pdfplumber>
- [23] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, “Multilingual e5 text embeddings: A technical report,” *arXiv preprint arXiv:2402.05672*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.05672>
- [24] intfloat. (2026) multilingual-e5-base. Model card officielle. [Online]. Available: <https://huggingface.co/intfloat/multilingual-e5-base>
- [25] ——. (2026) multilingual-e5-base – configuration. Configuration officielle du modèle. [Online]. Available: <https://huggingface.co/intfloat/multilingual-e5-base/blob/main/config.json>
- [26] Pinecone. (2026) Pinecone documentation. [Online]. Available: <https://docs.pinecone.io/guides/get-started/overview>
- [27] Nominatim Project. (2026) Nominatim api – overview. Documentation officielle. [Online]. Available: <https://nominatim.org/release-docs/latest/api/Overview/>
- [28] ——. (2026) Nominatim api – place output formats. Documentation officielle. [Online]. Available: <https://nominatim.org/release-docs/latest/api/Output/>
- [29] OpenStreetMap Foundation. (2026) Nominatim usage policy. Politique officielle d’utilisation du service Nominatim. [Online]. Available: <https://operations.osmfoundation.org/policies/nominatim/>

- [30] MLflow. (2026) Mlflow tracking. Documentation officielle MLflow. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking>
- [31] Helsinki-NLP. (2026) Helsinki-nlp/opus-mt-en-romance. Model card et configuration du modèle. [Online]. Available: <https://huggingface.co/Helsinki-NLP/opus-mt-en-ROMANCE>
- [32] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” *Proceedings of the 39th International Conference on Machine Learning*, 2022. [Online]. Available: <https://cdn.openai.com/papers/whisper.pdf>
- [33] OpenAI. (2026) Whisper model card. Documentation et modèle officiel Whisper. [Online]. Available: <https://github.com/openai/whisper/blob/main/model-card.md>
- [34] Qwen Team. (2025) Qwen3: Think deeper, act faster. Publication officielle Qwen. [Online]. Available: <https://qwenlm.github.io/blog/qwen3/>
- [35] Mistral AI. (2025) Mistral large 3. Documentation officielle Mistral AI. [Online]. Available: <https://docs.mistral.ai/models/mistral-large-3-25-12>
- [36] OpenAI. (2024) Gpt-4o. Documentation officielle OpenAI. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [37] Meta AI. (2024) Llama 3.3. Documentation officielle Meta AI. [Online]. Available: <https://ai.meta.com/resources/models-and-libraries/llama/>
- [38] ——. (2024) Llama 3.2. Documentation officielle Meta AI. [Online]. Available: <https://ai.meta.com/resources/models-and-libraries/llama/>
- [39] Google. (2026) Flutter documentation. [Online]. Available: <https://docs.flutter.dev/>
- [40] FastAPI. (2026) Fastapi documentation. [Online]. Available: <https://fastapi.tiangolo.com/tutorial/>
- [41] Meta AI. (2026) Faiss: A library for efficient similarity search and clustering of dense vectors. [Online]. Available: <https://github.com/facebookresearch/faiss>

- [42] SQLite. (2026) About sqlite. [Online]. Available: <https://www.sqlite.org/about.html>
- [43] Stripe. (2026) Stripe checkout documentation. [Online]. Available: <https://docs.stripe.com/payments/checkout>
- [44] —. (2026) How checkout works and webhooks. [Online]. Available: <https://docs.stripe.com/payments/checkout/how-checkout-works>
- [45] Docker. (2026) Running containers. [Online]. Available: <https://docs.docker.com/engine/containers/run>
- [46] GitHub. (2026) Github actions documentation. [Online]. Available: <https://docs.github.com/en/actions>
- [47] —. (2026) Using secrets in github actions. [Online]. Available: <https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>
- [48] Elastic. (2026) Explore logs with kibana. [Online]. Available: <https://www.elastic.co/guide/en/logs/guide/current/logs-app-overview.html>
- [49] ngrok. (2026) ngrok documentation. [Online]. Available: <https://ngrok.com/docs/getting-started/>
- [50] OpenRouter. (2026) Openrouter models api. URL vérifiée – redirige automatiquement vers la page canonique "list-all-models-and-their-properties". [Online]. Available: <https://openrouter.ai/docs/api/api-reference/models/get-models>



HONORIS UNITED UNIVERSITIES

École Supérieure Privée d'Ingénieurs de Monastir